INNOPOLIS UNIVERSITY

SNE MASTER TRACK

COURSE: [F25] SECURITY OF SYSTEMS AND NETWORKS

SSN Lab 2 Assignment

on

UEFI Secure Boot

Student Name: HASIB AL TAHSIN

Application ID: 187623

Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

Submission Date: 21.09.2025

Environment Setup

First, **Ubuntu 24.04.3 LTS** OS was installed in VMware Workstation Pro 17 virtual machine, but this version of virtual machine supports EFI BOOT. It does not support secure boot. I tried to enable secure boot manually or by using commands or .vmx file in notepad to forcefully, so that secure boot is disabled. **VMware 13++** versions support secure boot option easily. So later Ubuntu OS was used alongside Windows 11 OS manually.

After Setup, the device and OS info is shown in **Fig. 1.** The username is **luffy**, and this machine's name is **luff-vmware.**

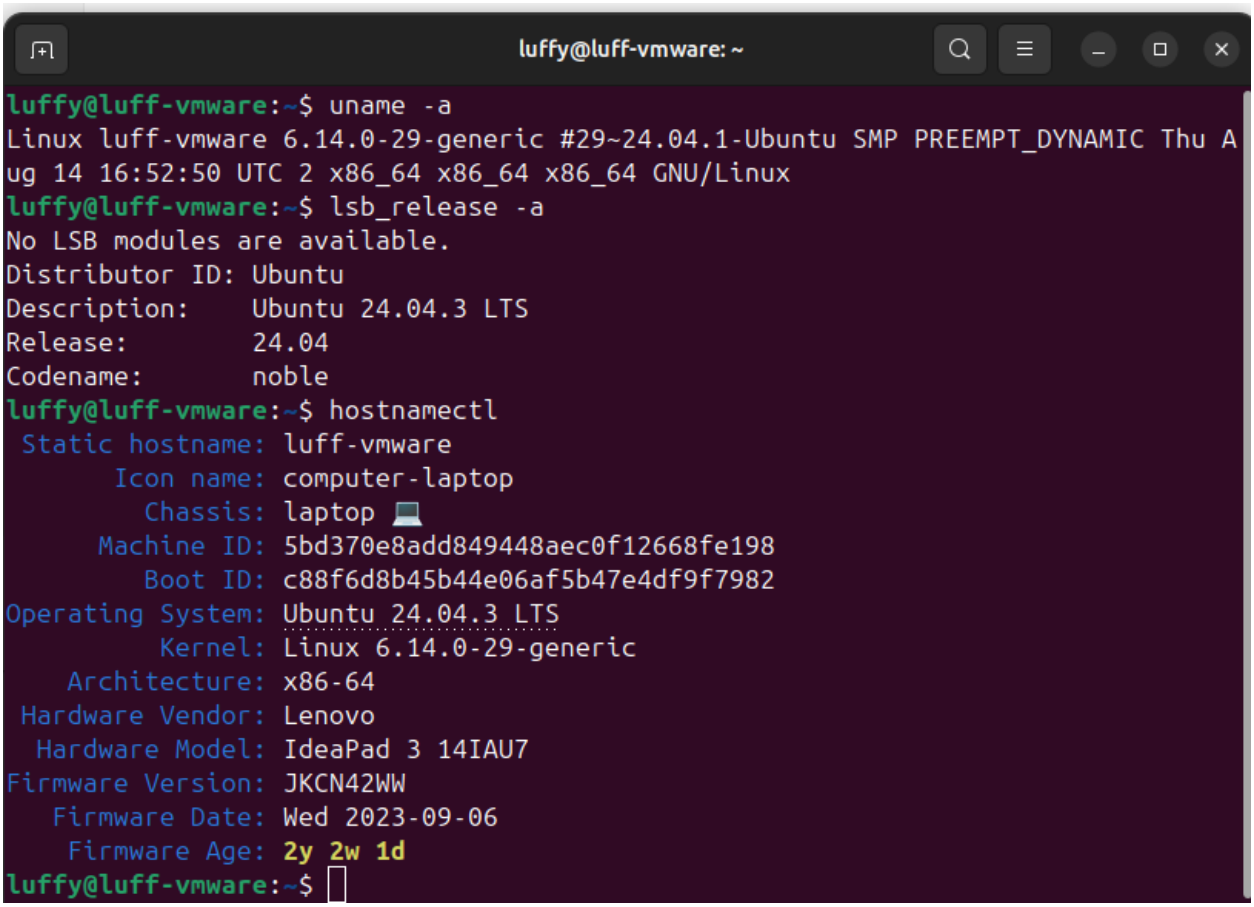


Fig. 1: Device and user info

Task 1 - Firmware Databases

Question 1: Extract the Microsoft certificate that belongs to the key referred to in Step 1 from the UEFI firmware, and show its text representation. Hint: efitools, openssl x509

2. Is this certificate the root certificate in the chain of trust? What is the role of the Platform Key (PK) and other variables?

Answer:

Part 1 - Extract the Microsoft certificate that belongs to the key referred to in Step 1 from the UEFI firmware, and show its text representation

Step A - Check Secure Boot status

Command:

mokutil --sb-state

Output:

Here we are going to make sure that Secure Boot is enabled **[1, 2]** and **in Fig. 2.**

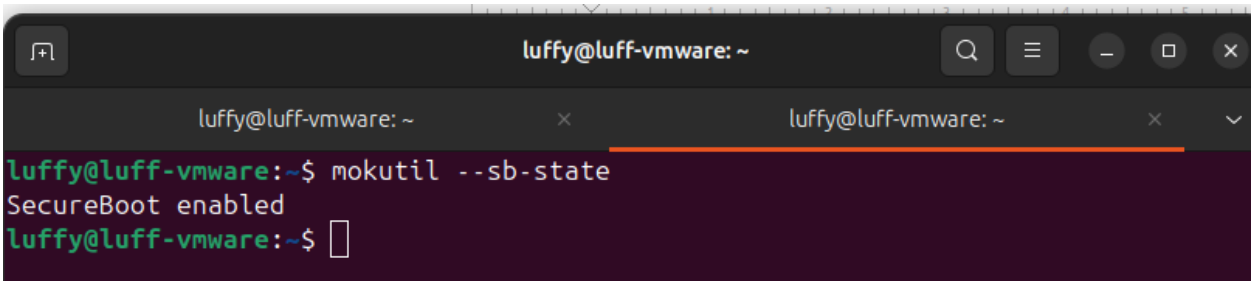


Fig. 2: Secure Boot is enabled in UEFI boot.

Step B - Check UEFI DB Variable

Command [3]:

sudo efi-readvar -v db

Output (Fig. 3):

The output in **Fig. 3** shows that Microsoft certificates are present in the DB. Here we have the required Microsoft UEFI CA 2011 certificate in **List 1.**

- **List 0 - Microsoft Windows Production PCA 2011**
- **List 1 - Microsoft Corporation UEFI CA 2011**
- **List 2 - Certificate_JK (Lenovo Certificate)**

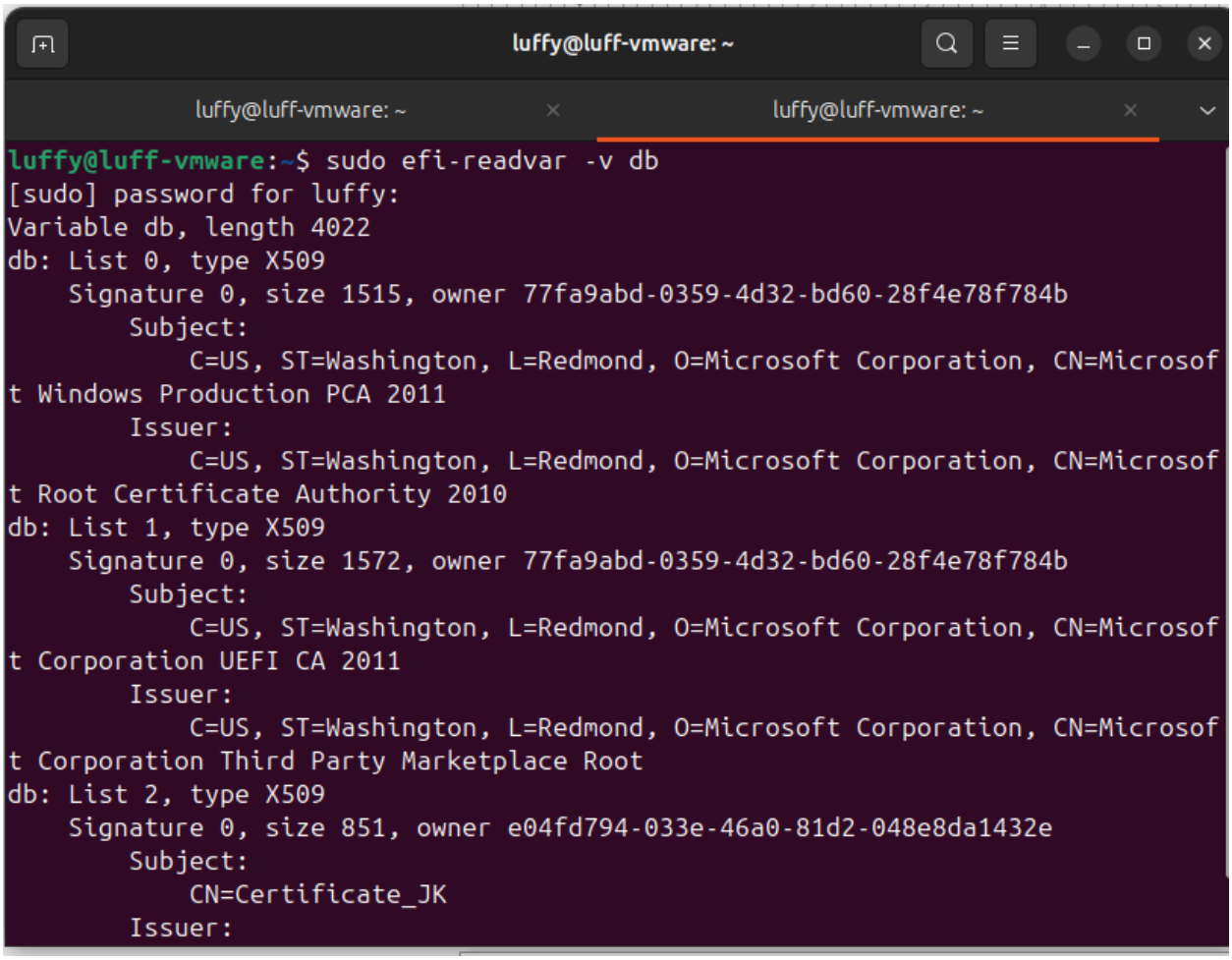


Fig. 3: Checking UEFI DB Variable.

Step C - Certificate Extraction (DER File)

Command [4]:

for f in ms-dbcert-*.der; do

```
  echo "---- $f ----"
  openssl x509 -in "$f" -inform DER -noout -subject -issuer
done
```

Output (Fig. 4):

It turns out that ms-dbcert-1.der is Microsoft Corporation UEFI CA 2011 **[4]** and all things are shown in **Fig. 4.**

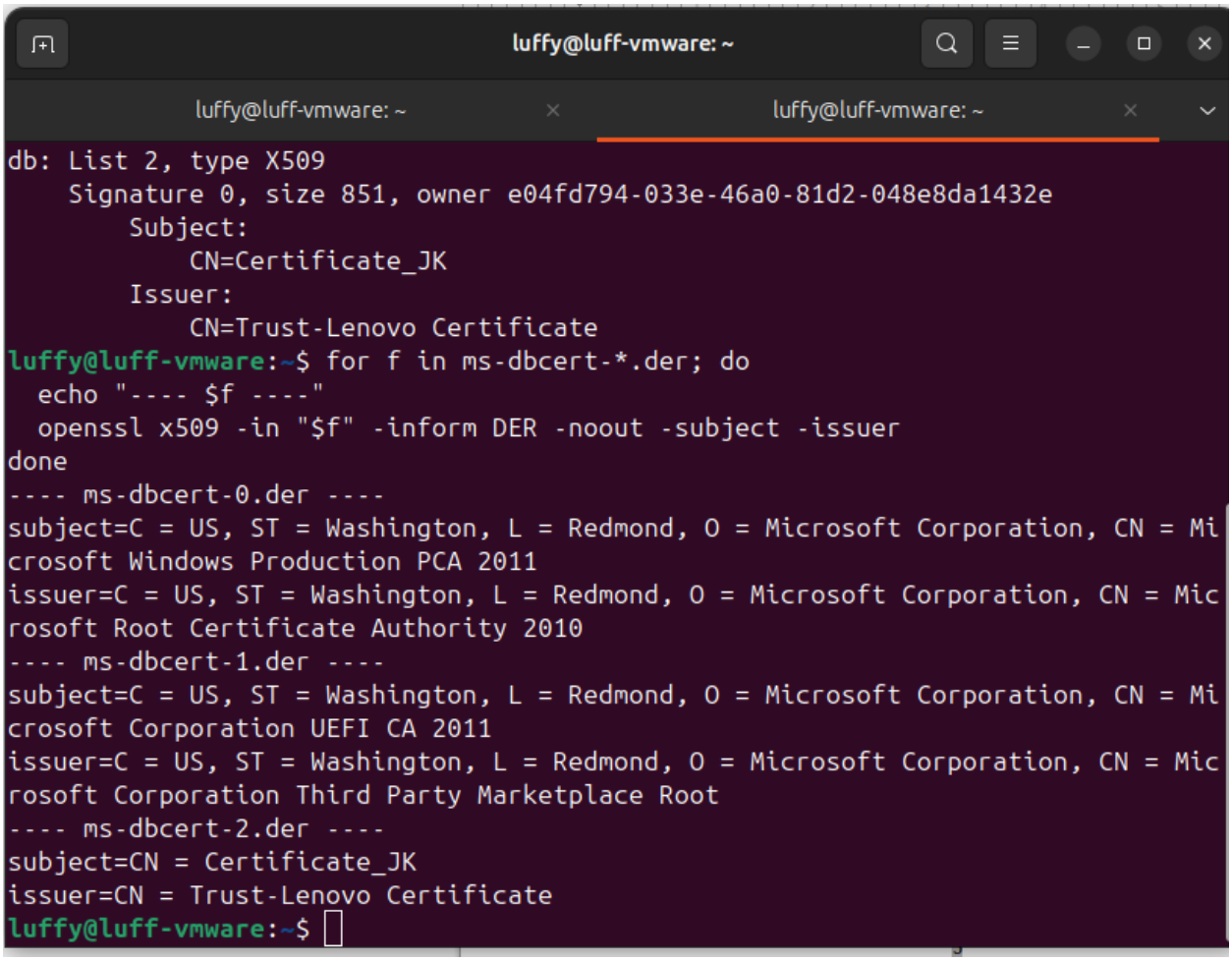


Fig. 4: Certificate Extraction (DER File)

Step D - Full Text Representation of Certificate

Command [5]:

```
openssl x509 -in ms-dbcert-1.der -inform DER -text -noout > ms_uefica.txt
```

less ms_uefica.txt

Output (Fig. 5 & 6):

The certificate is a CA certificate (CA: TRUE), valid until 2026, and is issued by a Microsoft Third Party Marketplace Root, and will expire in 2026 (Shown in Fig. . The Microsoft UEFI CA 2011 certificate (which is in the DB) is indeed valid until 2026 (usually 28 June 2026) **[4], [5], [9].**

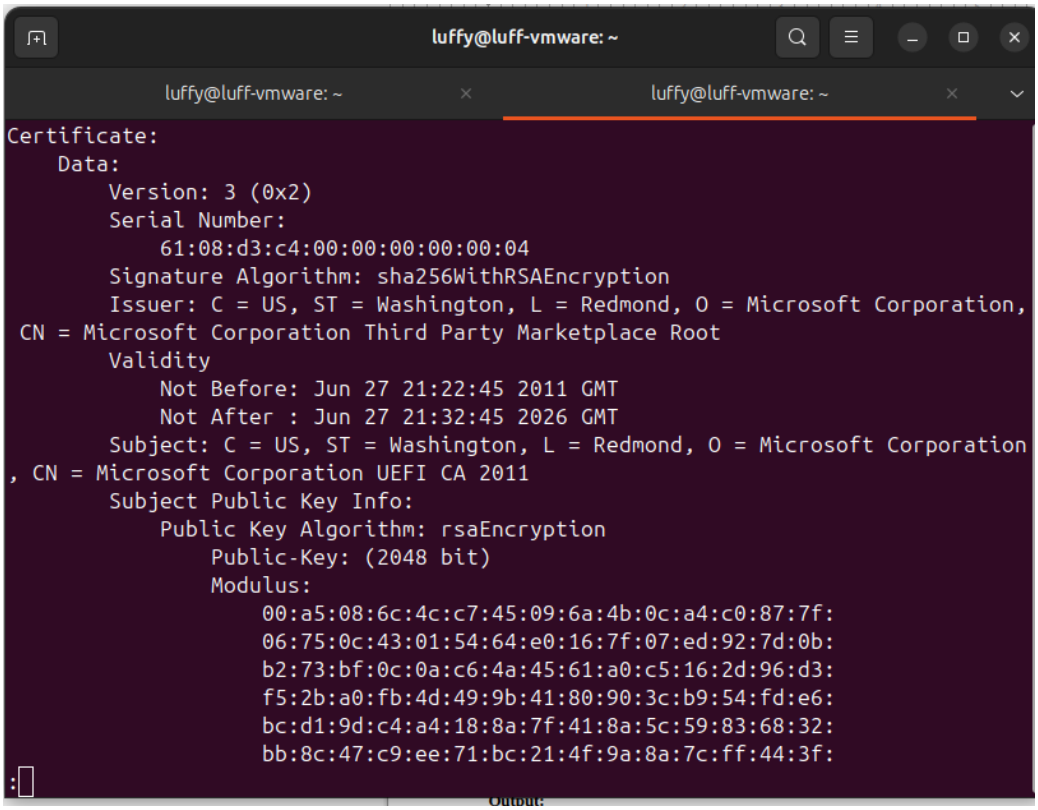


Fig. 5

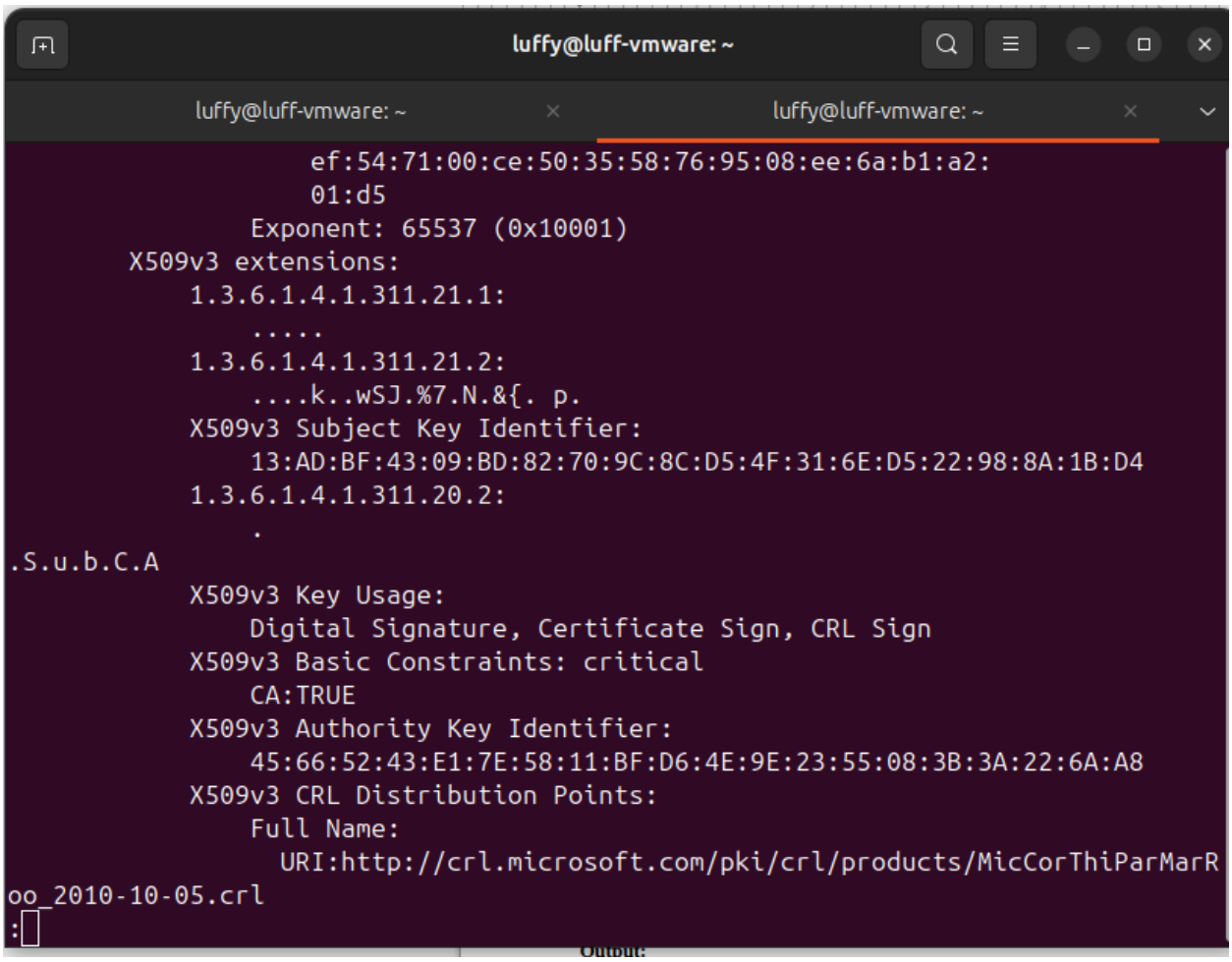


Fig. 6

Step E - PEM Conversion and Shim Verification

Command:

openssl x509 -in ms-dbcert-1.der -inform DER -out microsoft-uefica.pem -outform PEM

sbverify --cert microsoft-uefica.pem /boot/efi/EFI/ubuntu/shimx64.efi

Output (Fig. 7):

The output means that the Ubuntu Shim is indeed signed by the Microsoft UEFI CA certificate, and the signature verification was successful **[6].** The outputs are shown in **Fig. 7.**

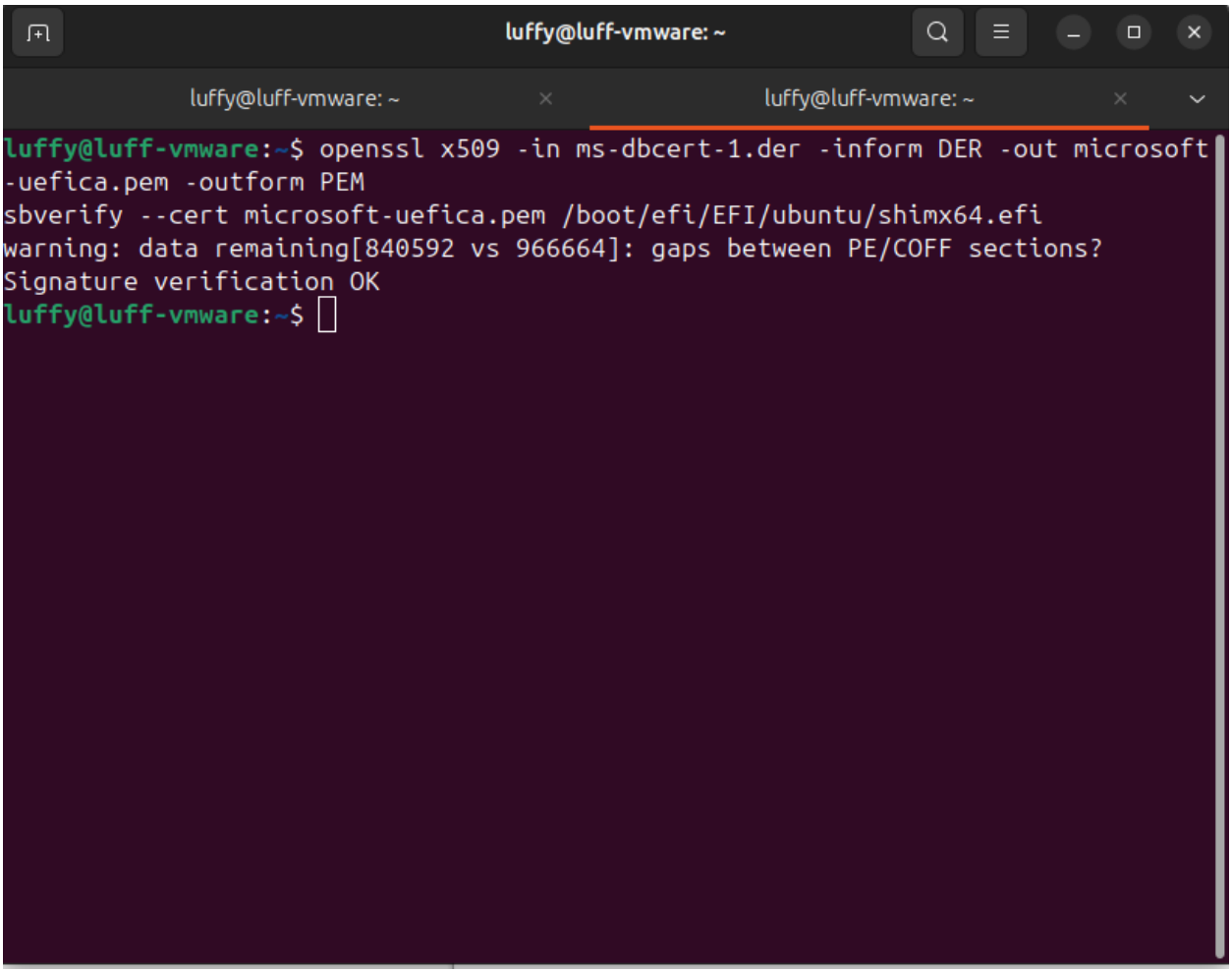


Fig. 7: PEM Conversion and Shim Verification

Part 2 - Is this certificate the root certificate in the chain of trust? What is the role of the Platform Key (PK) and other variables?

Answer:

Is it a Certificate Root

In Step 1, we extracted the Microsoft UEFI CA 2011 certificate from the ms-dbcert-1.der file.

Command [7]:

```
for f in ms-dbcert-*.der; do
  echo "---- $f ----"
```

```
  openssl x509 -in "$f" -inform DER -noout -subject -issuer
done
```

Output (Fig. 8):

ms-dbcert-1.der (Microsoft UEFI CA 2011) is an **Intermediate CA** and its Subject and Issuer are not the same. Its **Issuer = Microsoft Corporation Third Party Marketplace Root**, so it is not a **Root certificate**. If it were a root certificate, then **Subject = Issuer (self-signed root). ms-dbcert-1.der** is an **Intermediate CA, not a root certificate [4].** The outputs in this part are shown in **Fig. 8.**

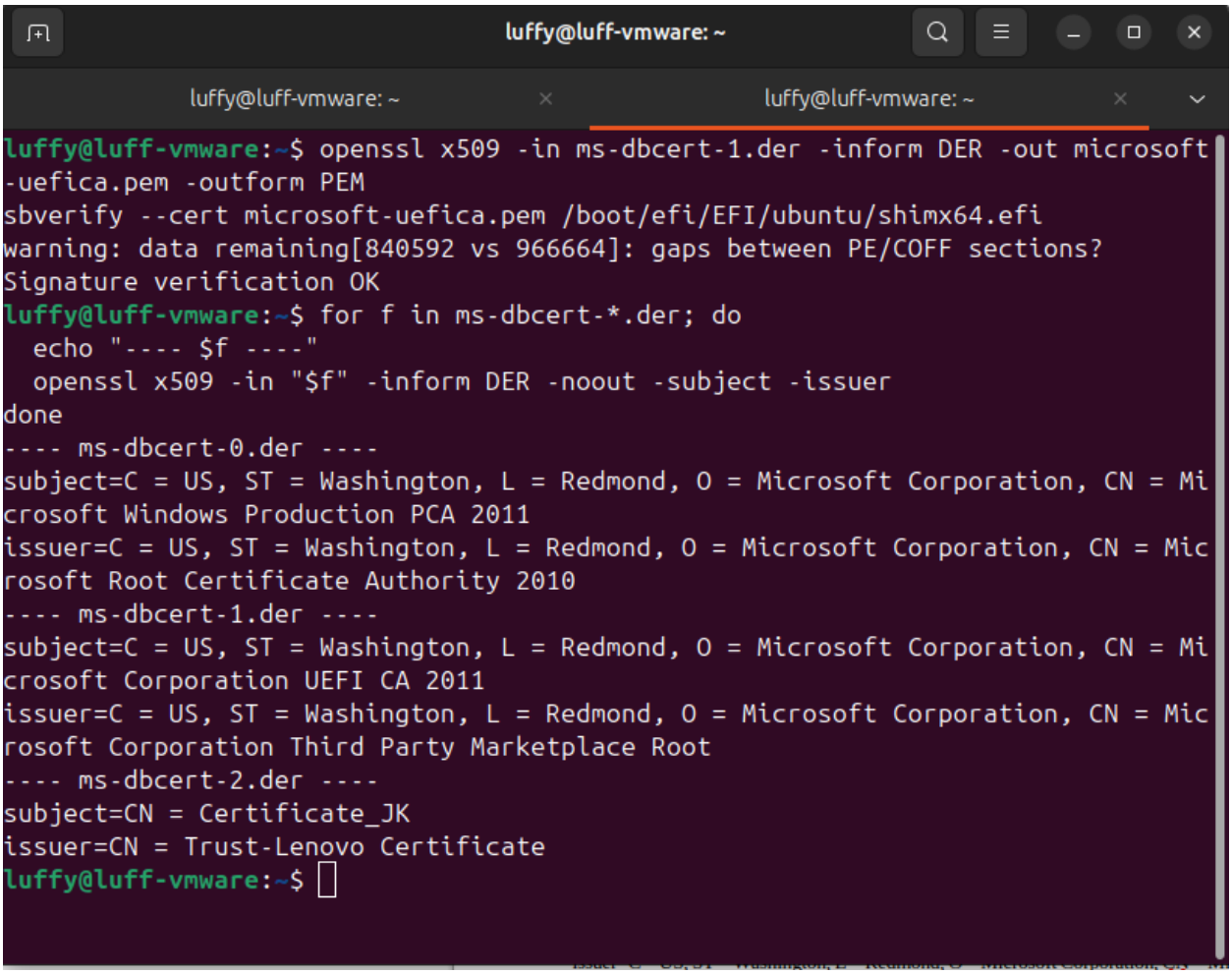


Fig. 8: Certificate Root and Role of PK

Role of the Platform Key (PK) and Other Variables

The role of PK, KEK, db, dbx, MOK from firmware variables has been verified with commands and outputs.

dbx (Forbidden Signatures – Blacklist):

Command:

sudo efi-readvar -v dbx

Output (Fig. 9):

A revoked bootloader certificate was found here, meaning compromised binaries will no longer run **[3].** The part of the output for this test is shown in **Fig. 9.**

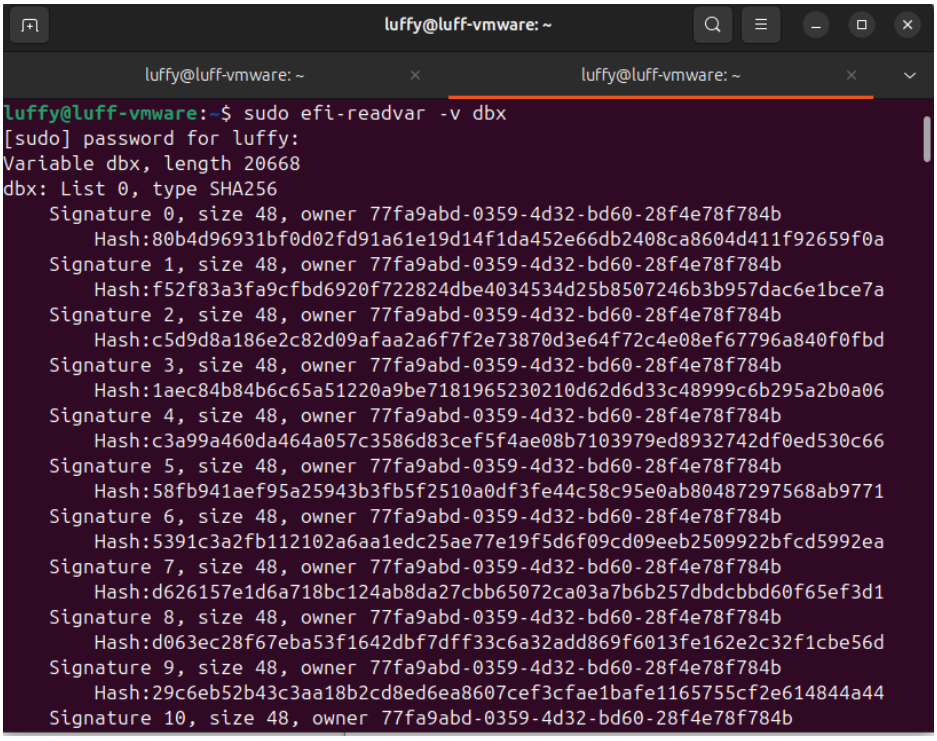


Fig. 9: The Part of the output of dbx

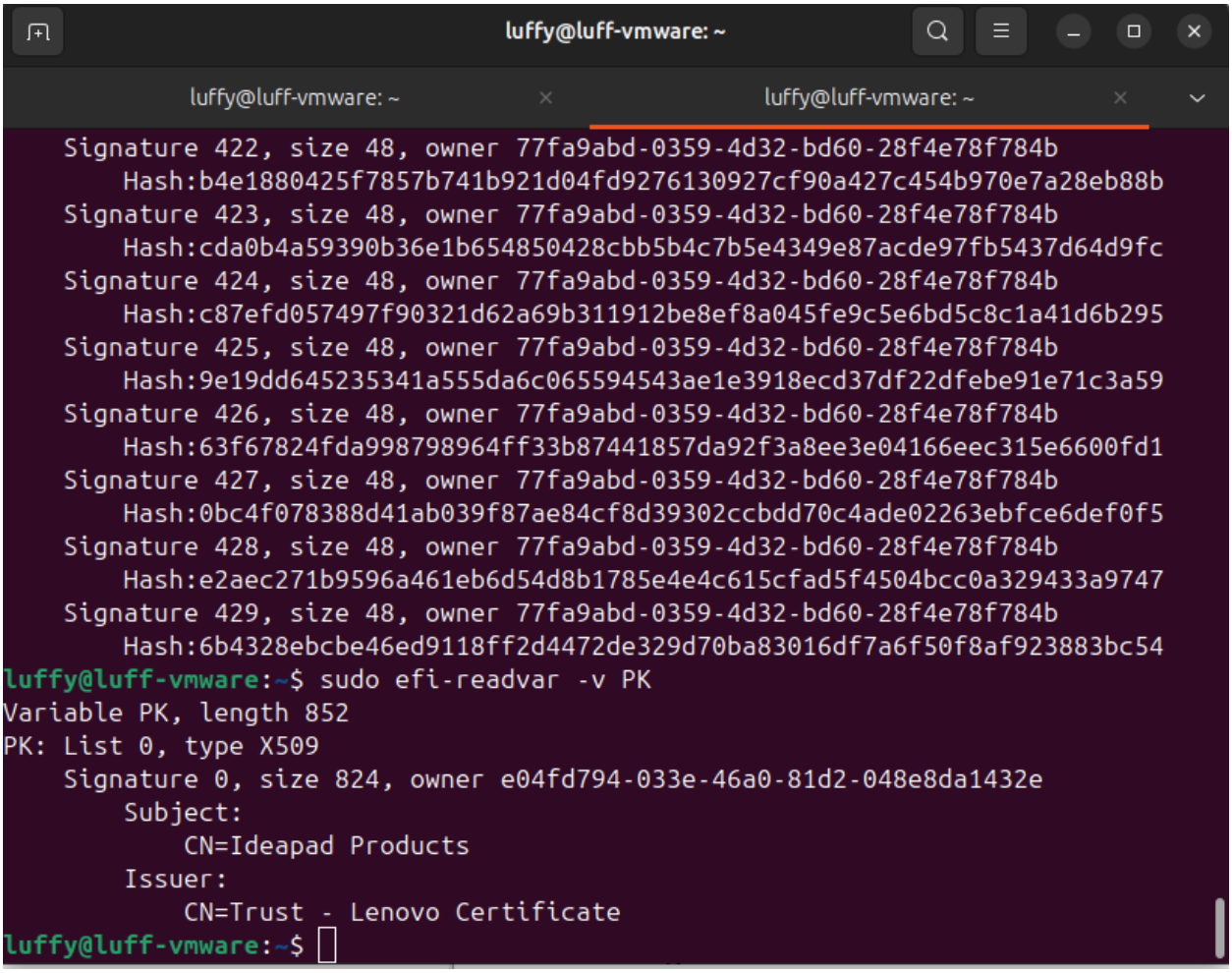


Fig. 10: The last part of the output of dbx and the starting part of PK.

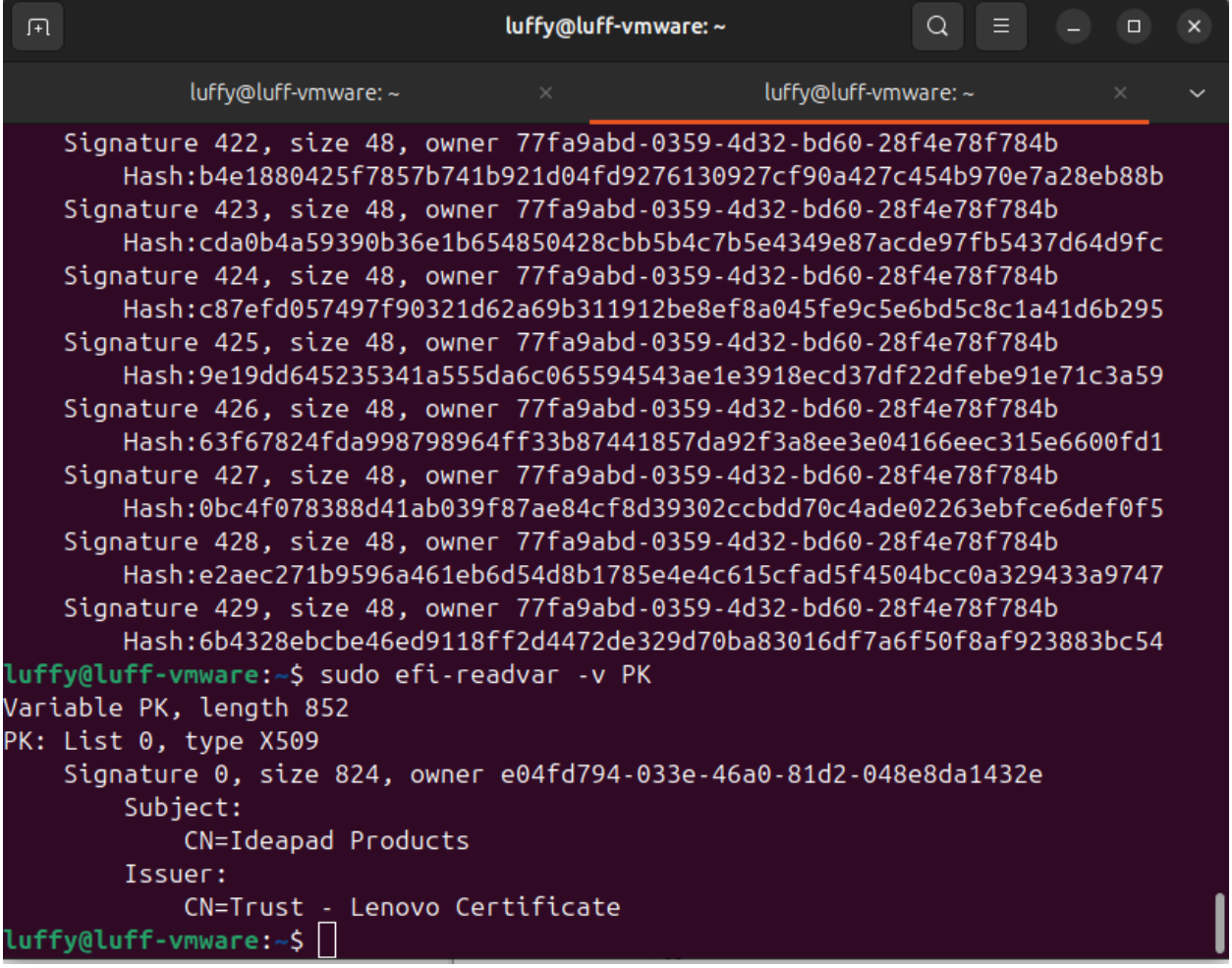
Platform Key (PK):

Role:

- Root of trust for firmware owner.
- KEK can be updated with PK.

Command:

sudo efi-readvar -v PK



Output (Fig. 10):

Fig. 10: Role of self-signed PK

PK self-signed, that is, the firmware owner's root key **[3]. Fig. 10** depicts the self-signed PK**.**

Key Exchange Key (KEK):

Role:

- Key authorized by PK.
- It is used to sign db/dbx updates.

Command:

sudo efi-readvar -v KEK

Output (Fig. 11):

The KEK contains the Microsoft KEK certificate, which is used to sign the db/dbx update **[3].**

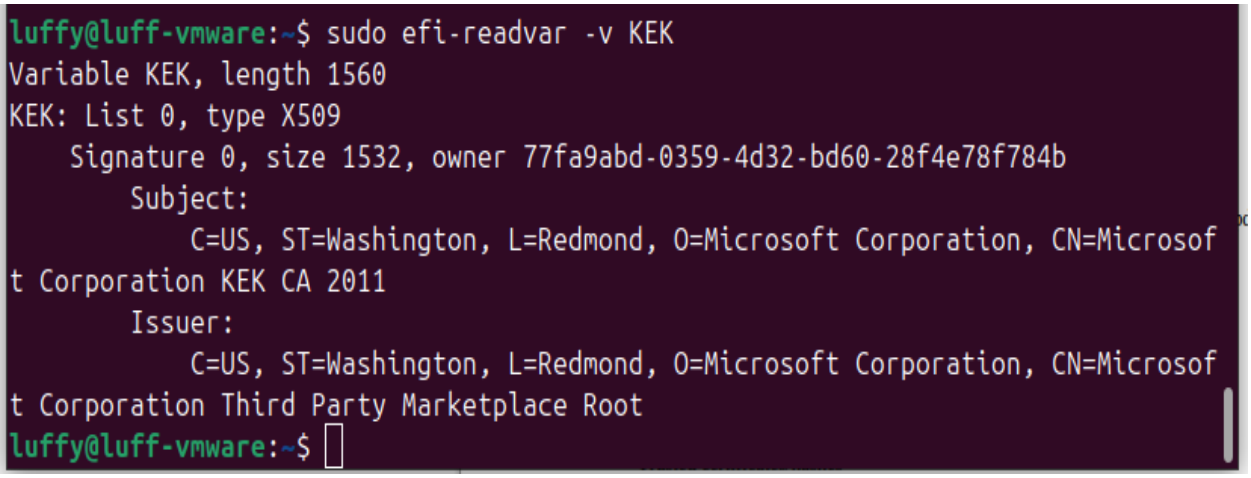


Fig. 11: The role of KEK

db (Allowed Signatures):

Role:

- Trusted certificates/hashes
- Allows loading of Shim/Grub.

Command:

sudo efi-readvar -v db

Output (Fig. 12):

It appears that Microsoft Corporation UEFI CA 2011 is present, which is required to verify the shim loader **[3].** The implementation of this command and it's output is shown in **Fig. 12.**

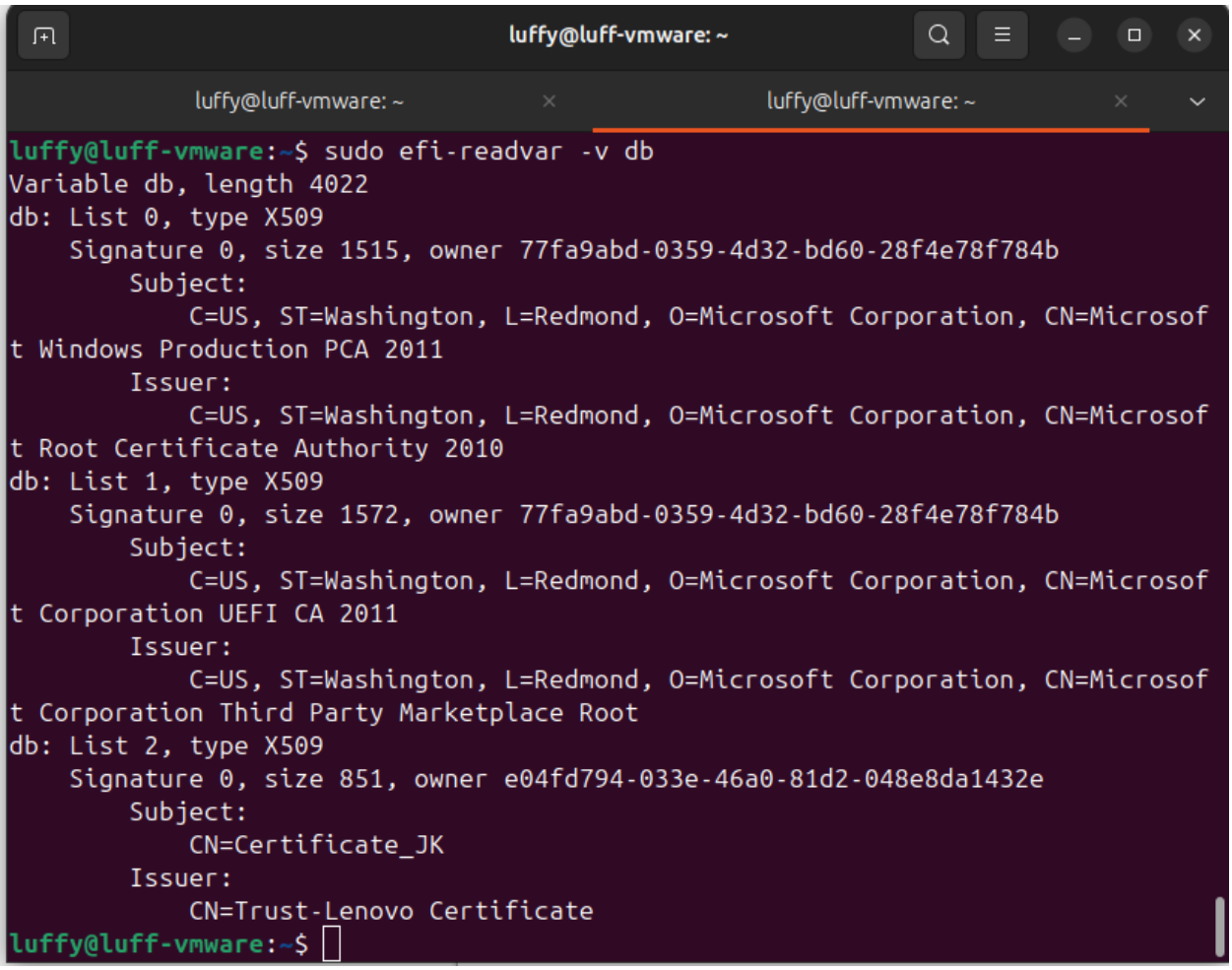


Fig. 12: The role of db.

MOK (Machine Owner Key):

Role:

- Shim's own key store.
- It is used to run user/distro custom kernel/module signing.

Command:

mokutil --list-enrolled

Output (Fig. 13 & 14):

Ubuntu's own signing key is found enrolled in this MOK **[3].** This is shown in **Fig. 13 and 14.**

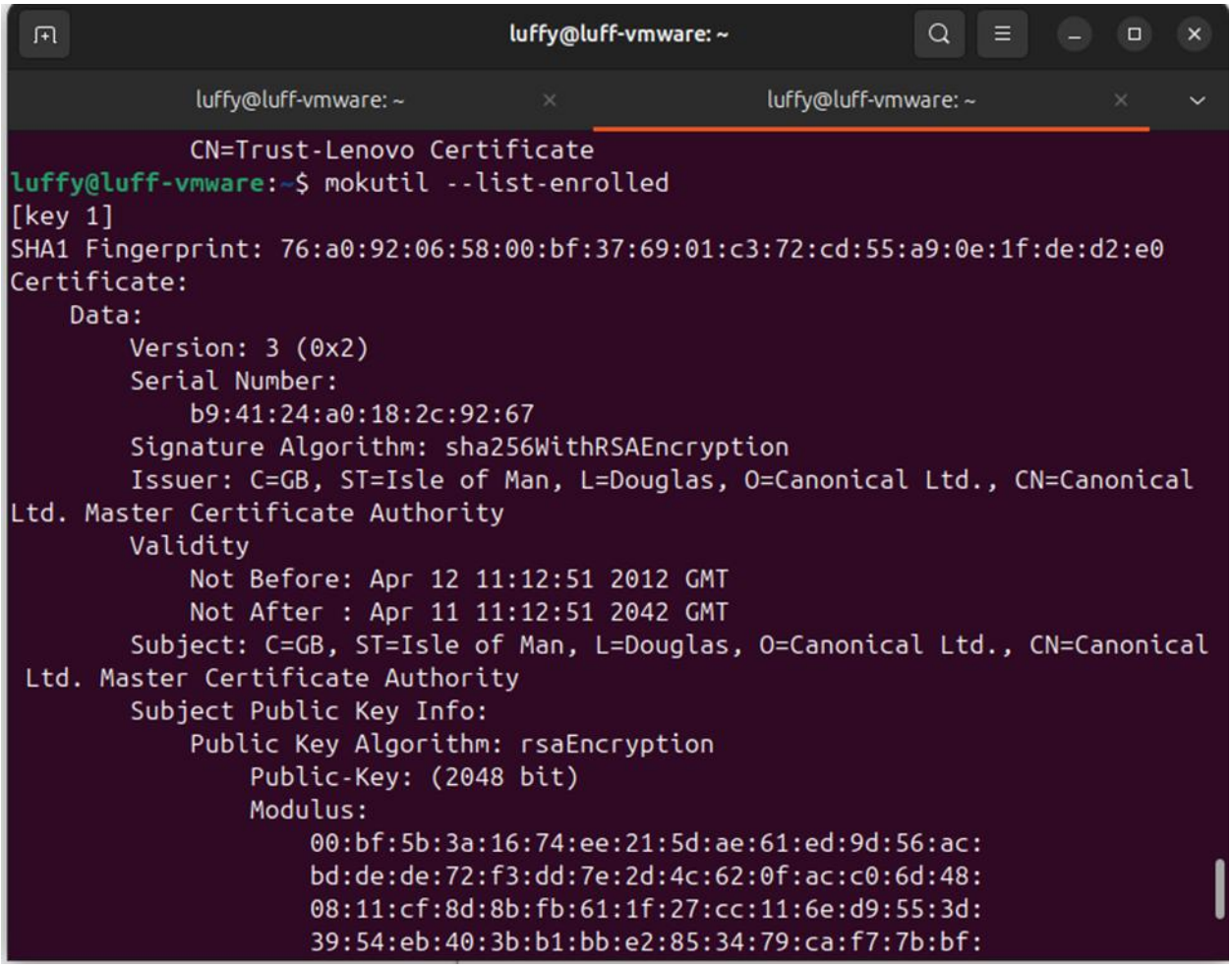


Fig. 13

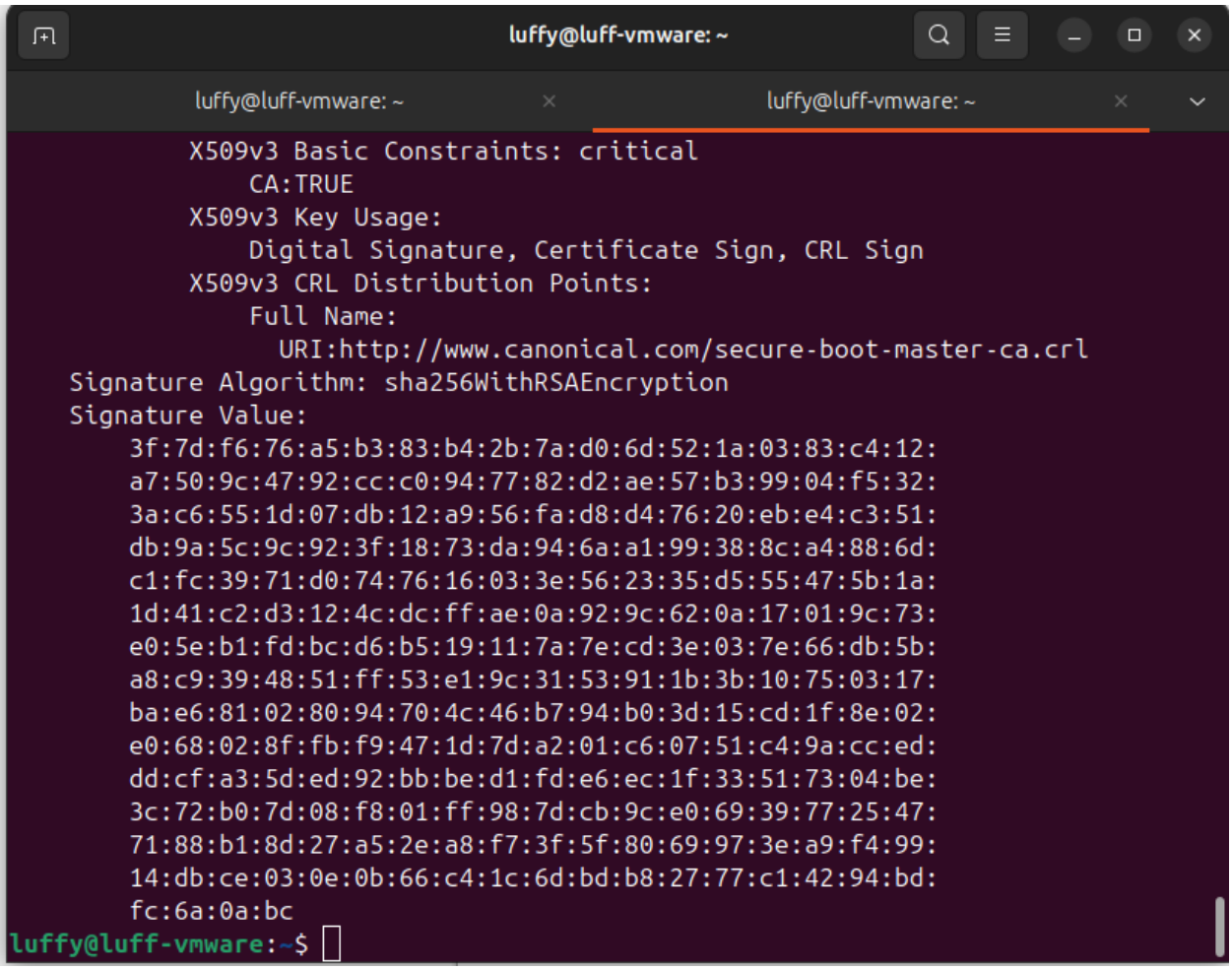


Fig. 14: The role of MOK.

Note:

- **PK** = Root key (self-signed).
- **KEK** = Microsoft KEK, db/dbx update allowed.
- **db** = Allowed signatures (with Microsoft UEFI CA).
- **dbx** = Revoked list (blacklisted certs).
- **MOK** = key store of Shim (Ubuntu custom key enrolled).

Summary - Task 1

1. mokutil --sb-state = Secure Boot enabled.
2. efi-readvar -v db = Microsoft Corporation UEFI CA 2011 is in the DB.
3. openssl x509 -text = The certificate is from the CA, not the root.

4. **sbverify** = Shim is indeed signed by Microsoft UEFI CA (OK).
5. ms-dbcert-1.der (Microsoft UEFI CA 2011) is not a root certificate, it is an Intermediate CA, because its **Issuer = Microsoft Corporation Third Party Marketplace Root**.
6. **PK** = Firmware owner's root key.
7. **KEK** = PK signs authorized keys, db/dbx updates.
8. **db** = allowed signatures (where Microsoft UEFI CA 2011 is present).
9. **dbx** = revoked signatures blacklist
10. **MOK** = shim's custom key store (Ubuntu signing key enrolled here).

Firmware DB → **Microsoft UEFI CA 2011** → **Shim** → **Grub** → **Kernel** chain-of-trust worked correctly **[3], [4], [6].**

Task 2 - SHIM

3. Verify that the system indeed boots the shim boot loader in the first stage. What is the full path name of this bootloader?

Hint: efibootmgr

4. Verify that the shim bootloader is indeed signed with the "Microsoft Corporation UEFI CA" key.

Hint: sbsigntool, PEM format, pesign, sbverify

UEFI binaries such as OS/boot loaders, drivers and kernels on Linux are executables in the

Windows Authenticode Portable Executable Signature Format. This format is described on

https://www.symbolcrash.com/wp-content/uploads/2019/02/Authenticode_PE-1.pdf. On this page you can download version 1.0 of the specification of the format. Read the first 9 pages of the specification (up to "Authenticode-Specific Structures"). Focus on the structure of the binaries.

5. What is the exact name of the part of the binary where the actual signature is stored?

6. In what standard cryptographic format is the signature data stored?

To extract the signature data from the binary, one needs to determine the exact location and size of this data in the binary. This information is stored in the Data Directories section of the Optional Header of the executable, as shown in Figure 1 of the specification. To retrieve location and size you can used the pyew package, as follows:

- Start pyew from the command line with the full name of the shim binary as argument.

- Wait for the pyew prompt to appear (it will do an analysis on the binary which you can interrupt with Ctrl-C if it takes too long)

- Type pyew.pe.OPTIONAL HEADER.DATA DIRECTORY to get a listing of all data directory entries.

- Locate the one called IMAGE DIRECTORY ENTRY SECURITY. The location and size are given as the “VirtualAddress:” and “Size:” fields.

7. Extract the signature data from the shim binary using dd. Add 8 bytes to the location as given in the data directory to skip over the Microsoft WIN CERTIFICATE structure header (see page 14 of the specification if you are interested).

Hint: pyew.py, -/+8.

8. Show the subject and issuer of all X.509 certificates stored in the signature data. Draw a diagram relating these certificates to the “Microsoft Corporation UEFI CA” certificate.

Hint: binwalk

Now we know that the system indeed boots the shim, and that this OS loader is indeed signed by Microsoft and where this signature is stored.

Answer:

Part 3 of Task 2: Verify shim bootloader path:

Command [8]:

efibootmgr -v

Output (Fig. 15 & 16):

The part of Output is shown in **Fig. 15 & 16.**

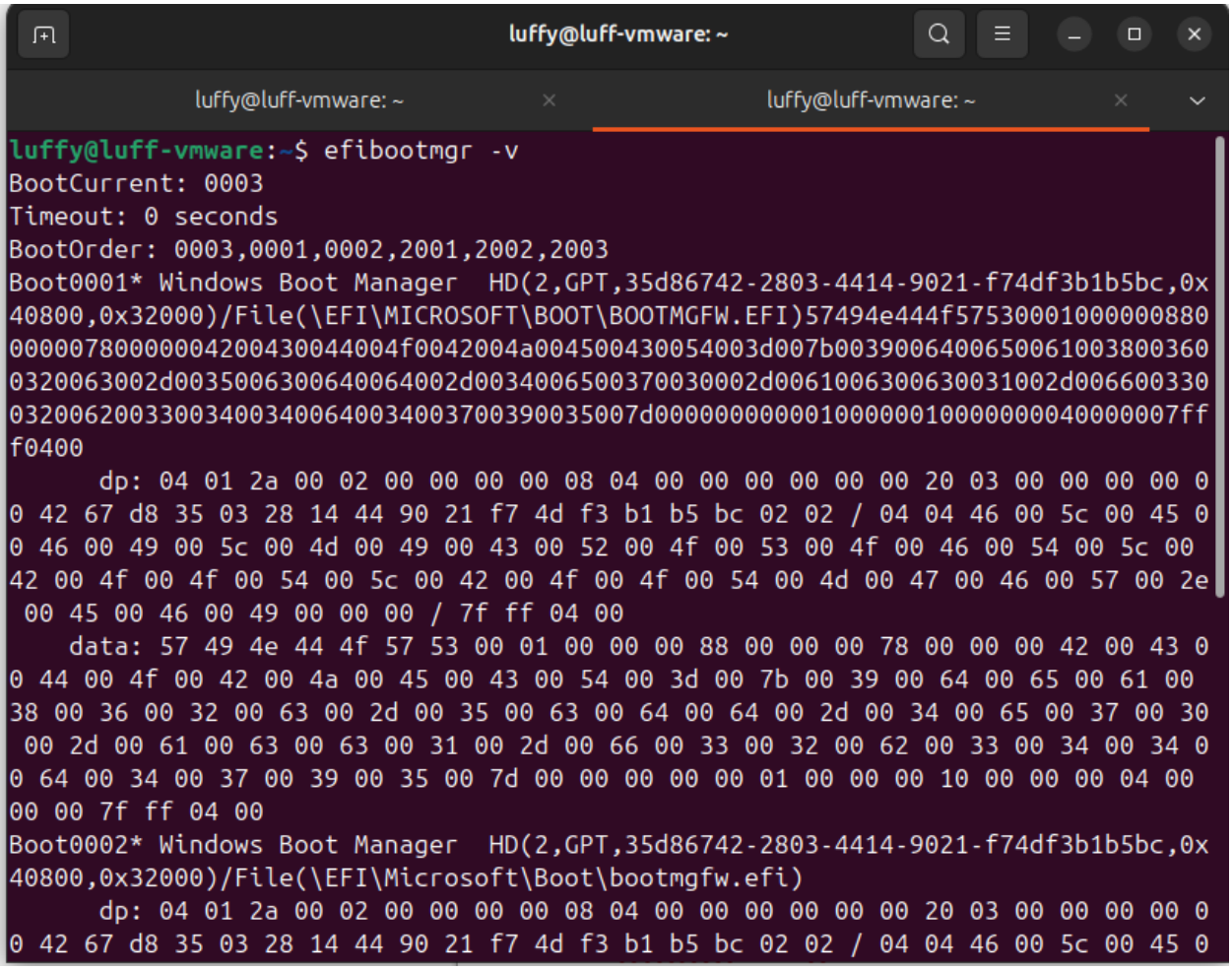


Fig. 15: Verify shim bootloader path part 1

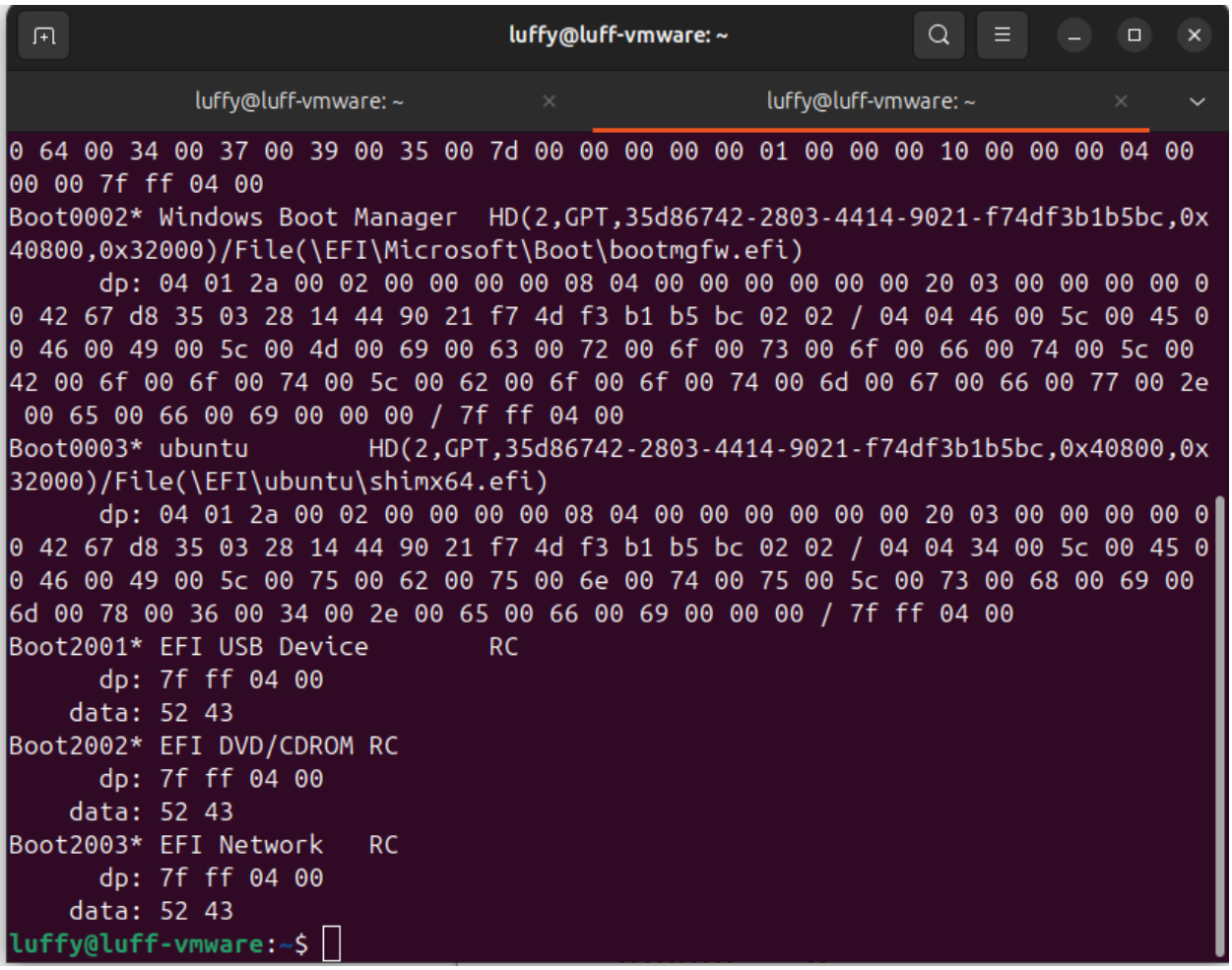


Fig. 16: Verify shim bootloader path last part

Full path of shim bootloader:

/boot/efi/EFI/ubuntu/shimx64.efi

Part 4 of Task 2: Verify shim is signed by Microsoft UEFI CA

Command [9]:

openssl x509 -in ms-dbcert-1.der -inform DER -text -noout

Output (Fig. 16, 17):

Some parts of the result are shown in **Fig. 16 & 17.**

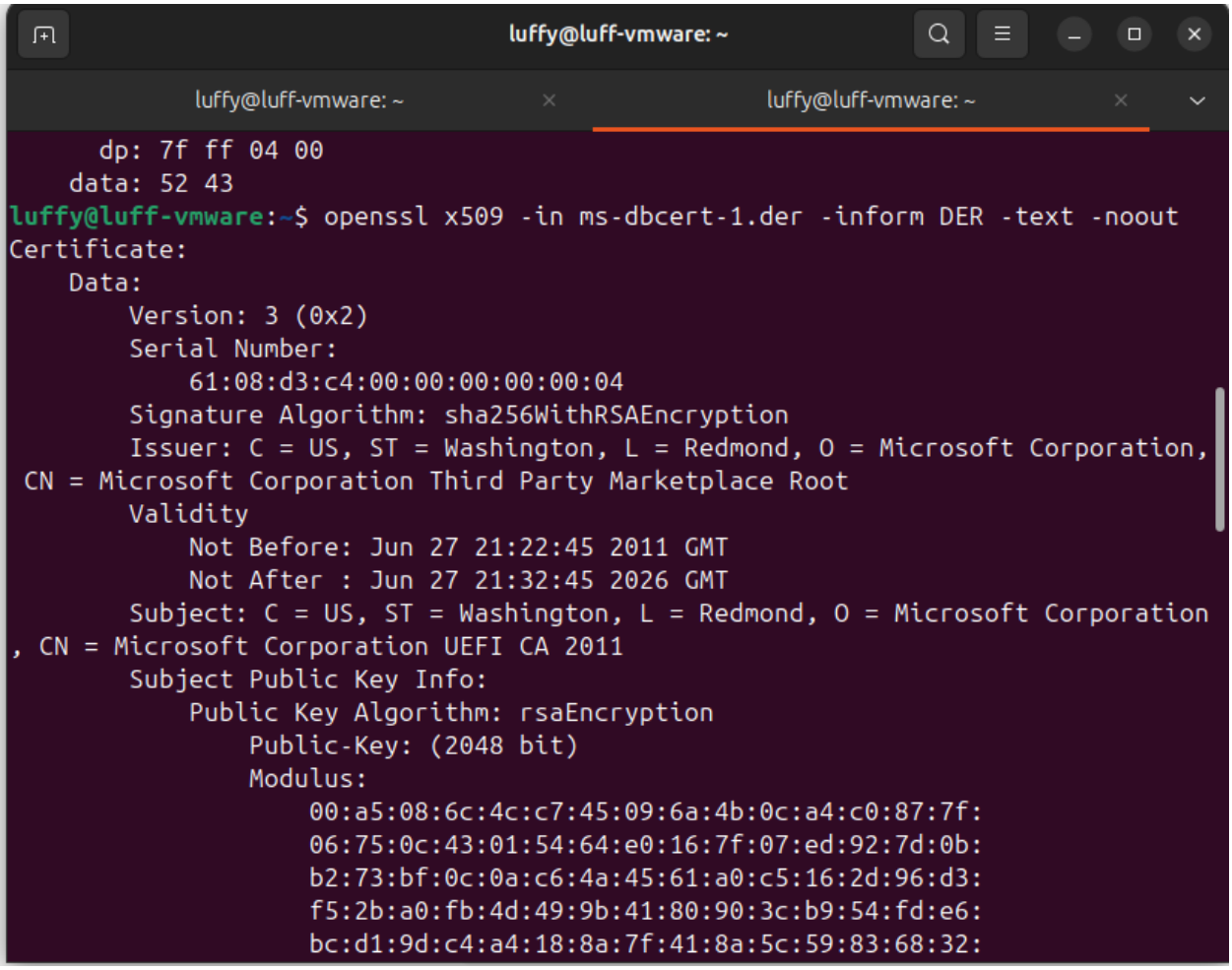


Fig. 16: Verify shim is signed by Microsoft UEFI CA **part 1**

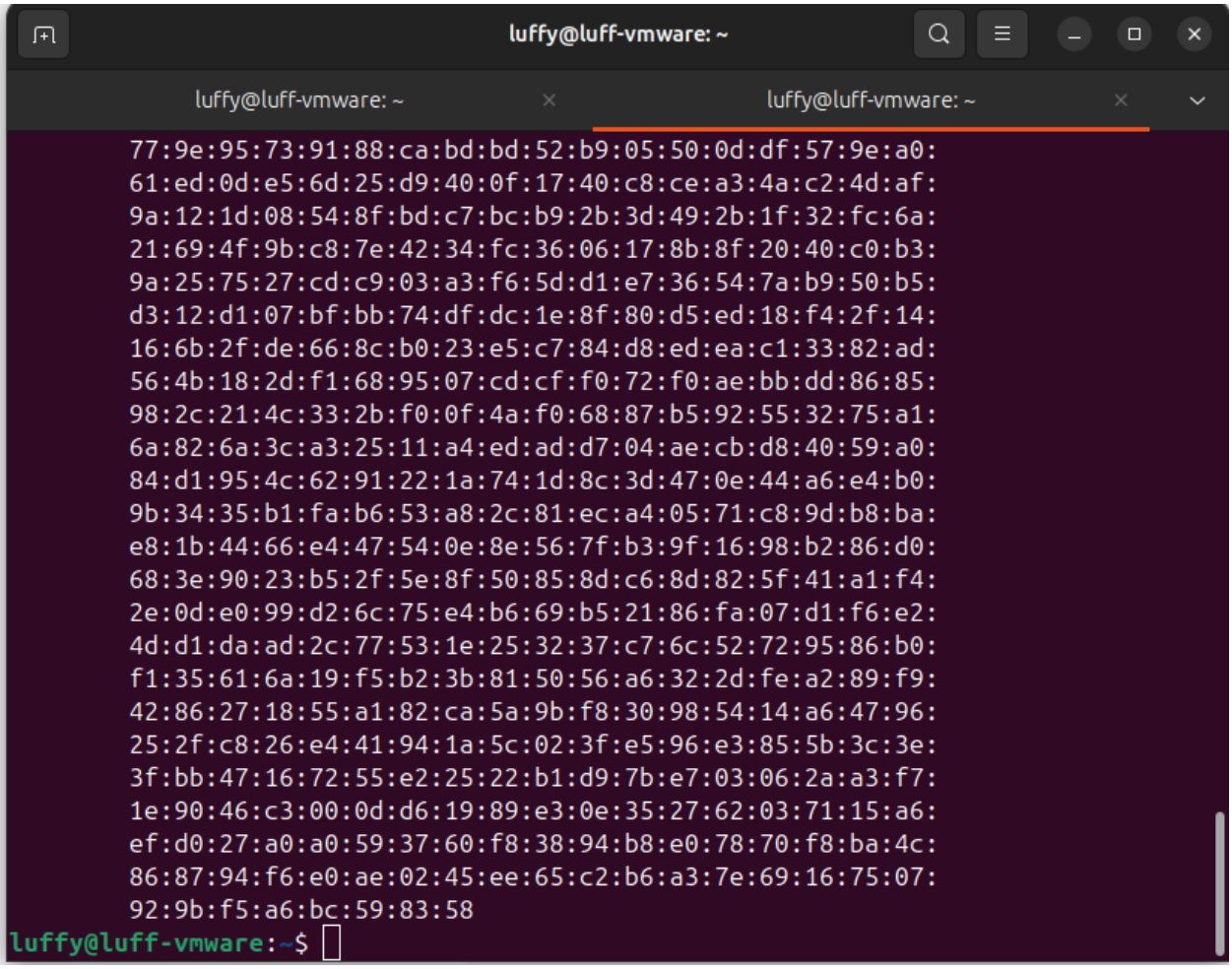


Fig. 17: Verify shim is signed by Microsoft UEFI CA **last part**

The output shows that the certificate has been issued:

Issuer: Microsoft Corporation Third Party Marketplace Root

Subject: Microsoft Corporation UEFI CA 2011

Verification with sbverify:

sbverify --cert microsoft-uefica.pem /boot/efi/EFI/ubuntu/shimx64.efi

Output (Fig. 18):

This signature verification result is shown in **Fig. 18.**

```
luffy@luff-vmware:~$ sbverify --cert microsoft-uefica.pem /boot/efi/EFI/ubuntu/s
himx64.efi
warning: data remaining[840592 vs 966664]: gaps between PE/COFF sections?
Signature verification OK
luffy@luff-vmware:~$
```

Fig. 18: Verification with sbverify is done.

It is confirmed that Shim is signed by *Microsoft Corporation UEFI CA 2011*.

Part 5 of Task 2: Where the signature is stored

Command:

Sudo apt install pesign

pesign -i /boot/efi/EFI/ubuntu/shimx64.efi -h

Output (Fig. 19):

The signature's location is /boot/efi/EFI/ubuntu/shimx64.efi which is shown in **Fig. 19.**

```
Processing triggers for man-db (2.12.0-4build2) ...
luffy@luff-vmware:~$ pesign -i /boot/efi/EFI/ubuntu/shimx64.efi -h
724de6844dd0fe618ba5776c7bca0728be38a6544e24e44ef259b987b7abce80 /boot/efi/EFI/u
buntu/shimx64.efi
luffy@luff-vmware:~$
```

Fig. 19: Location of the Signature.

In UEFI binaries (PE/COFF format), the signature is stored in a specific entry in the Optional Header → Data Directory.

Exact Part:

IMAGE_DIRECTORY_ENTRY_SECURITY (called Security Directory) →

WIN_CERTIFICATE.bCertificate

Command:

objdump -x /boot/efi/EFI/ubuntu/shimx64.efi | grep -A5 "Security Directory"

The signature of the Shim binary is always stored in this Certificate Table section **[10-11] by using** objdump. The **Fig. 20** is shown where the signature is stored using objdump.

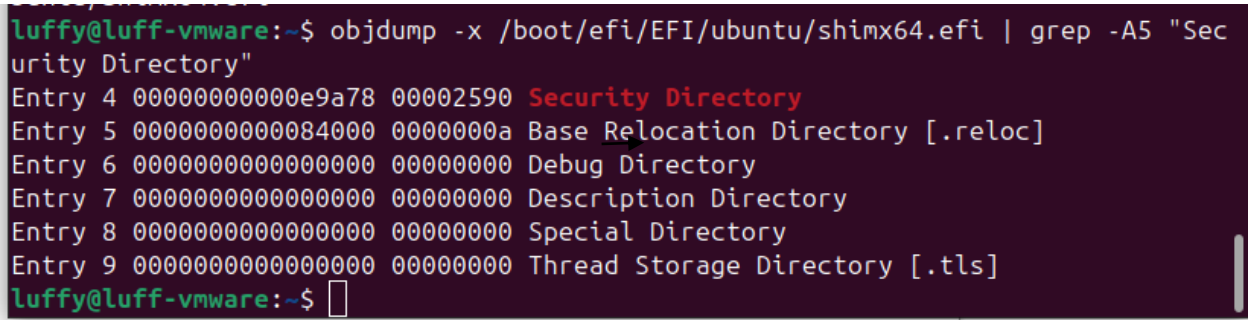


Fig. 20: The location of stored signature using objdjump.

Part 6 of Task 2: Signature is in Cryptographic Format

Signature data is stored (List signatures with pesign):

Format:

PKCS#7 (Cryptographic Message Syntax, CMS)

- DER encoded
- Contains one or more X.509 certificates

Command:

esign -S -i /boot/efi/EFI/ubuntu/shimx64.efi

Output (Fig. 21):

The signature of the Shim binary is always stored in this Certificate Table section **[10-11]** and it's also shown in **Fig. 21.**

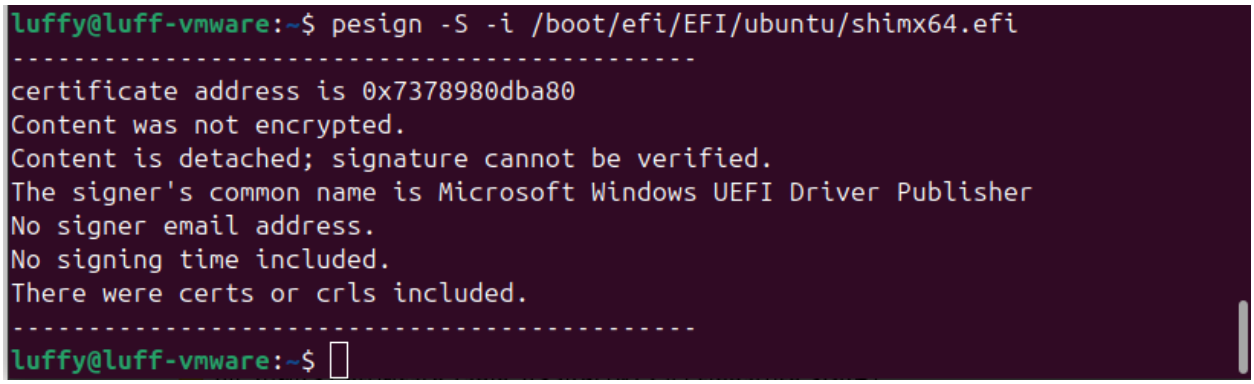


Fig. 21: List signatures with pesign

Part 7 of Task 2: Extracting Signature Data from Shim Binary

Command sequence [10] (to VirtualAddress and Hex**):**

```
hexVA=$(objdump -x /boot/efi/EFI/ubuntu/shimx64.efi | awk '/Security Directory/ {print $2}')
hexSize=$(objdump -x /boot/efi/EFI/ubuntu/shimx64.efi | awk '/Security Directory/ {print $3}')
```

Output (fig. 22):

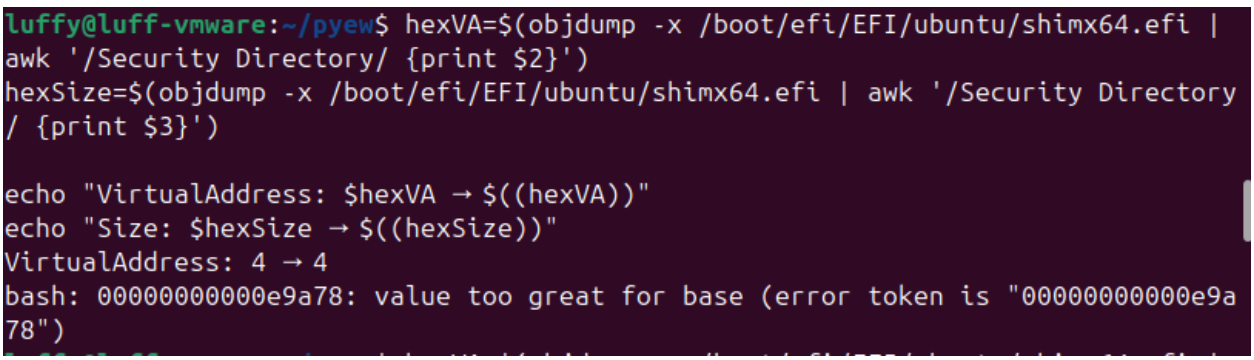


Fig. 22

Command 2 (Fig. 23) [10]:

```
hexVA=$(objdump -x /boot/efi/EFI/ubuntu/shimx64.efi | awk '/Security Directory/ {print $2}')

hexSize=$(objdump -x /boot/efi/EFI/ubuntu/shimx64.efi | awk '/Security Directory/ {print $3}')

VA_DEC=$(printf "%d\n" $((16#${hexVA#0*})))

SIZE_DEC=$(printf "%d\n" $((16#${hexSize#0*})))

echo "VirtualAddress: $hexVA → $VA_DEC"

echo "Size: $hexSize → $SIZE_DEC"
```

Output (Fig. 23):



Fig. 23

Hex to Decimal conversion (Fig. 24) [10]:

Command:

```
# Hex of Security Directory from objdump
hexVA=$(objdump -x /boot/efi/EFI/ubuntu/shimx64.efi | awk '/Security Directory/ {print $2}')
hexSize=$(objdump -x /boot/efi/EFI/ubuntu/shimx64.efi | awk '/Security Directory/ {print $3}')

# Adding leading zeros and 0x prefix
hexVA_fmt="0x$(echo $hexVA | sed 's/^0*//')"
hexSize_fmt="0x$(echo $hexSize | sed 's/^0*//')"

# Decimal conversion using Bash arithmetic
VA_DEC=$((16#${hexVA_fmt#0x}))
SIZE_DEC=$((16#${hexSize_fmt#0x}))

# Formatted output
echo "Hex to Decimal conversion:"
echo "  • VirtualAddress = $hexVA_fmt → $VA_DEC"
echo "  • Size = $hexSize_fmt → $SIZE_DEC"
```

Output (Fig. 24):

```
luffy@luff-vmware:~/pyew$ # Hex of Security Directory from objdump
hexVA=$(objdump -x /boot/efi/EFI/ubuntu/shimx64.efi | awk '/Security Directory/
{print $2}')
hexSize=$(objdump -x /boot/efi/EFI/ubuntu/shimx64.efi | awk '/Security Directory
/ {print $3}')

# Adding leading zeros and 0x prefix
hexVA_fmt="0x$(echo $hexVA | sed 's/^0*//')"
hexSize_fmt="0x$(echo $hexSize | sed 's/^0*//')"

# Decimal conversion using Bash arithmetic
VA_DEC=$((16#${hexVA_fmt#0x}))
SIZE_DEC=$((16#${hexSize_fmt#0x}))

# Formatted output
echo "Hex to Decimal conversion:"
echo "  • VirtualAddress = $hexVA_fmt → $VA_DEC"
echo "  • Size = $hexSize_fmt → $SIZE_DEC"
Hex to Decimal conversion:
  • VirtualAddress = 0x4 → 4
  • Size = 0xe9a78 → 957048
luffy@luff-vmware:~/pyew$
```

Fig. 24

dd command (for cutting signature data):

sudo dd if=/boot/efi/EFI/ubuntu/shimx64.efi of=/tmp/shim.pkcs7 \

bs=1 skip=$((917504+8)) count=$((107008-8)) status=progress

Output (Fig. 25):

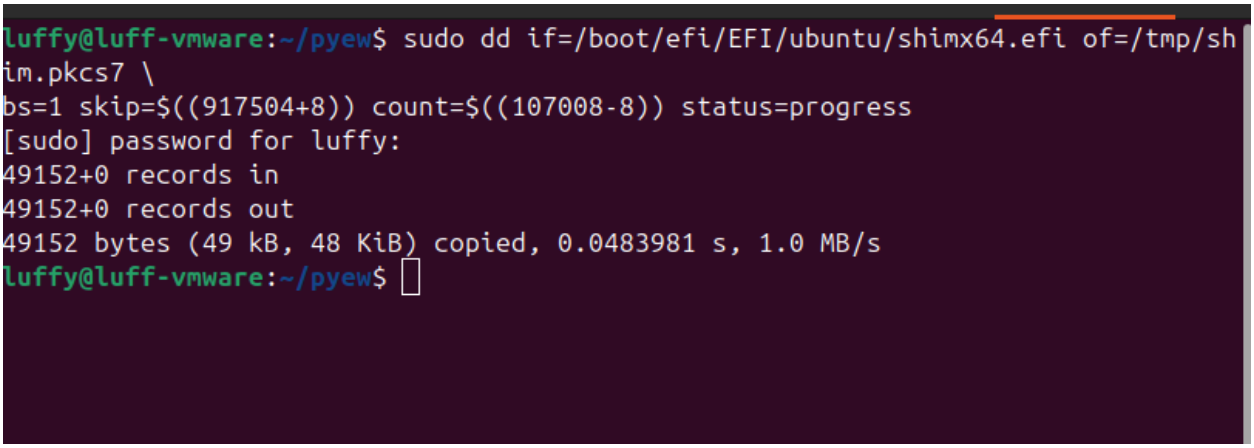


Fig. 25

File type verification:

file /tmp/shim.pkcs7

Output (Fig. 26):

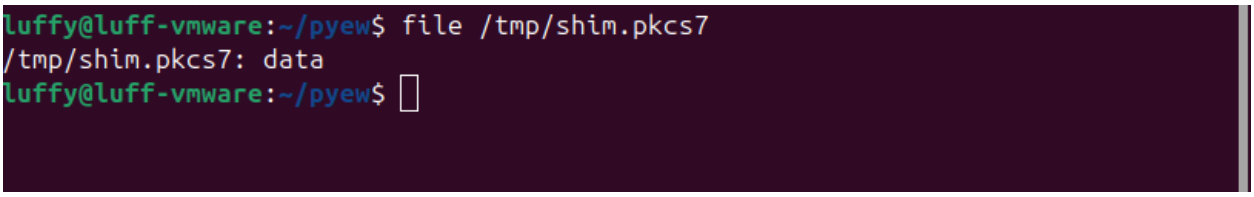


Fig. 26

This confirmed that the Signature portion was successfully extracted from the Shim binary and that it was DER Encoded PKCS#7.

Part 8 of Task 2: Extracting Certificates from Shim Signature Data

Command [11]:

Using sbverify: Command:

sudo apt install sbsigntools

sbverify --list /boot/efi/EFI/ubuntu/shimx64.efi

Output (Fig. 27):

```
luffy@luff-vmware:~/pyew$ sudo apt install sbsigntools
sbverify --list /boot/efi/EFI/ubuntu/shimx64.efi
[sudo] password for luffy:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
E: Unable to locate package sbsigntools
warning: data remaining[840592 vs 966664]: gaps between PE/COFF sections?
signature 1
image signature issuers:
 - /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft Corporatio
n UEFI CA 2011
image signature certificates:
 - subject: /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft W
indows UEFI Driver Publisher
   issuer:  /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft C
orporation UEFI CA 2011
 - subject: /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft C
orporation UEFI CA 2011
   issuer:  /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft C
orporation Third Party Marketplace Root
luffy@luff-vmware:~/pyew$ sbverify --list /boot/efi/EFI/ubuntu/shimx64.efi | gre
p -E 'subject:|issuer:'
warning: data remaining[840592 vs 966664]: gaps between PE/COFF sections?
 - subject: /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft W
```

Fig. 27

Subject/issuer filter (sbverify output) Fig. 28:

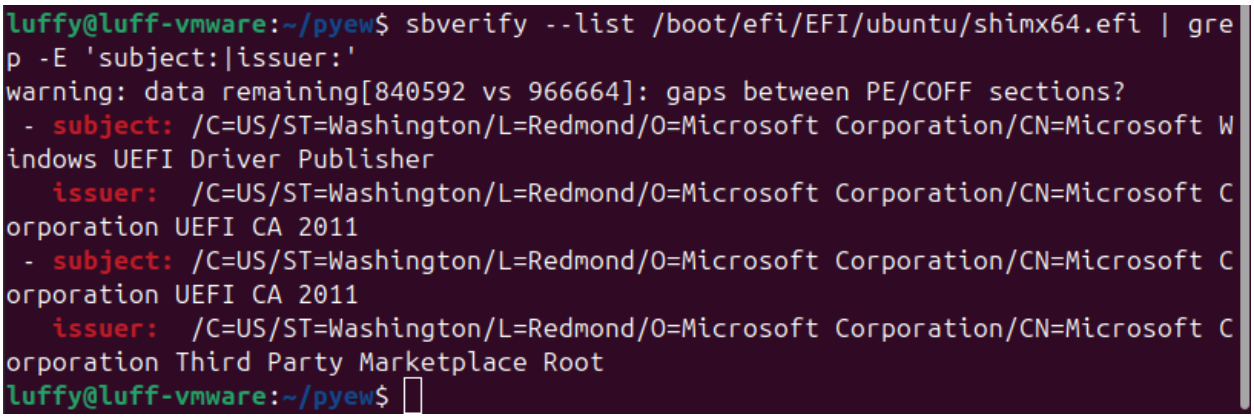


Fig. 28

Command:

sbverify --list /boot/efi/EFI/ubuntu/shimx64.efi | grep -E 'subject:|issuer:' | sed -E 's/^[[:space:]]*subject: /SUBJECT: /; s/^[[:space:]]*issuer: /ISSUER: /'

Output (Fig. 29):

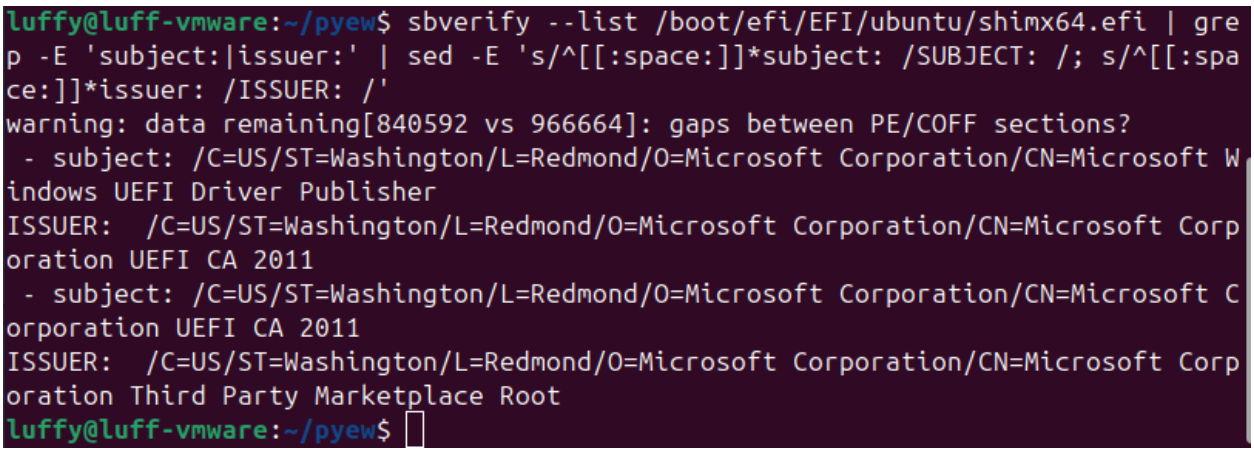


Fig. 29

Certificate Chain Diagram:

Output:

Microsoft Corporation Third Party Marketplace Root



Microsoft Corporation UEFI CA 2011



Microsoft Windows UEFI Driver Publisher (shim signer)

This output shows that the shimx64.efi binary is actually signed by Microsoft Windows UEFI Driver Publisher, which is under the Microsoft Corporation UEFI CA 2011, and that CA is issued by the Microsoft Third Party Marketplace Root **[11].**

Task 3 - GRUB

In Step 2 of the Ubuntu description, it says that "the second stage bootloader (grub-efiamd64-signed) is signed with Canonical's "Canonical Ltd. Secure Boot Signing" key." This boot loader binary is called grubx64.efi and is located in the same directory as the shim binary.

9. Using your new knowledge about Authenticode binaries, extract the signing certificates from the GRUB boot loader, and show the subject and issuer. You now know that the GRUB binary is indeed signed, but how does shim verify this signature? To do so shim needs a certificate that it trusts and which must not be modifiable by an external party without detection. The solution chosen by Ubuntu is to store a certificate in the shim binary. We will call this certificate X.

10. Why is storing the certificate X in the shim binary secure?

11. What do you think is the subject CommonName (CN) of this X certificate?

12. Obtain the X certificate used by the shim to verify the GRUB binary. There are two ways to obtain it: from the source code or from the binary directly. Hints for the latter case: - The certificate is in X.509 DER/ASN.1 format (see openssl asn1parse) - DER/ASN.1 leaves the CommonName readable - The certificate is 1080 bytes long - binwalk/dd

13. Verify that this X certificate's corresponding private key was indeed used to sign the GRUB binary.

Answer:

Part 9 of Task 3: Extract signing certificates from GRUB boot loader

Command [12]:

ls -lh /boot/efi/EFI/ubuntu/grubx64.efi

Output [Fig. 30]:

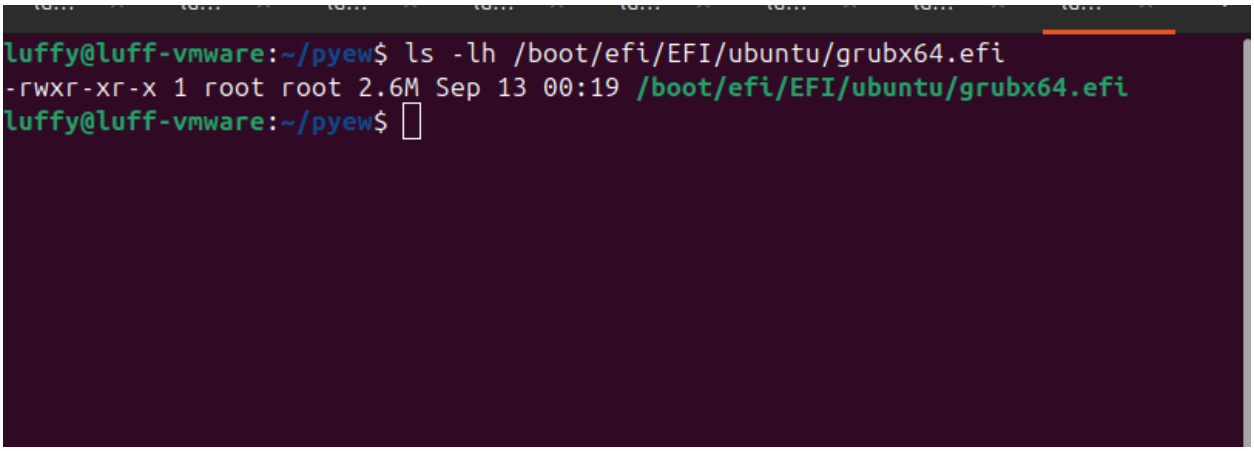


Fig. 30

Command [12]:

sudo sbverify --list /boot/efi/EFI/ubuntu/grubx64.efi

Output (Fig. 31):

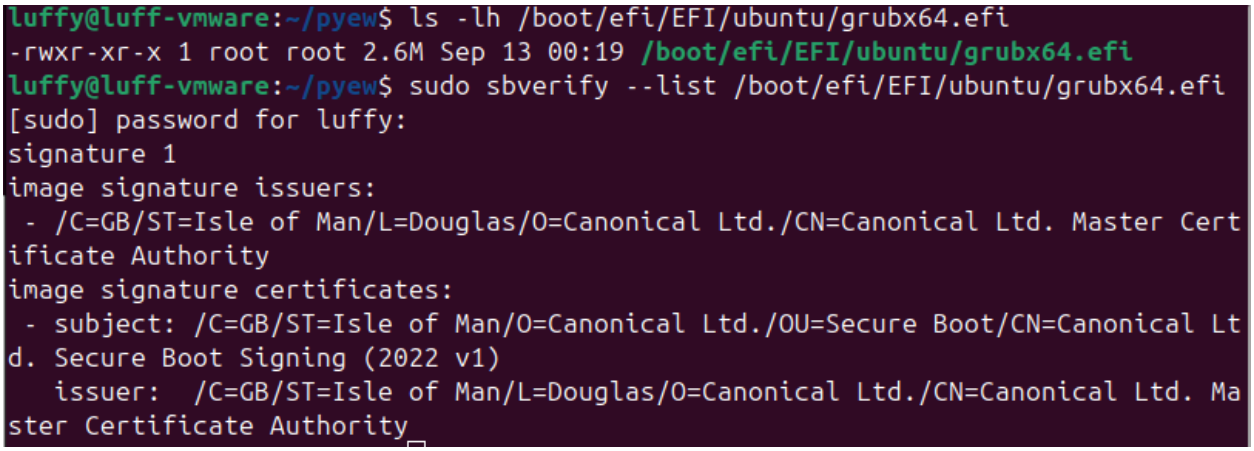


Fig. 31

Part 10 of Task 3: Is storing certificate X in shim secure or not

The Shim binary itself is signed by the Microsoft UEFI CA and the Microsoft certificate is trusted in the Secure Boot db variable. So, the certificate X inside the Shim cannot be modified by an external attacker - because modifying the Shim would invalidate the signature and Secure Boot would detect it. So, it is safe to have the certificate X embedded **[4].**

Part 11 of Task 3: Subject CN of certificate X

Certificate X is actually Canonical's Secure Boot Signing certificate. Its subject CN will be **[4]:**

Canonical Ltd. Secure Boot Signing (2022 v1)

Part 12 of Task 3: Obtain certificate X from shim binary **[13]**

Command:

sudo binwalk -e /boot/efi/EFI/ubuntu/shimx64.efi

Output (Fig. 32, 33):

```
luffy@luff-vmware:~/pyew$ sudo binwalk -e /boot/efi/EFI/ubuntu/shimx64.efi

Extractor Exception: Binwalk extraction uses many third party utilities, which m
ay not be secure. If you wish to have extraction utilities executed as the curre
nt user, use '--run-as=root' (binwalk itself must be run as root).
------------------------------------------------------------------------------
------------------
Traceback (most recent call last):
  File "/usr/lib/python3/dist-packages/binwalk/core/module.py", line 258, in __i
nit__
    self.load()
  File "/usr/lib/python3/dist-packages/binwalk/modules/extractor.py", line 147,
in load
    raise ModuleException("Binwalk extraction uses many third party utilities, w
hich may not be secure. If you wish to have extraction utilities executed as the
 current user, use '--run-as=%s' (binwalk itself must be run as root)." % user_i
nfo.pw_name)
binwalk.core.exceptions.ModuleException: Binwalk extraction uses many third part
y utilities, which may not be secure. If you wish to have extraction utilities e
xecuted as the current user, use '--run-as=root' (binwalk itself must be run as
root).
------------------------------------------------------------------------------
------------------
```

Fig. 32

```
Extractor Exception: Binwalk extraction uses many third party utilities, which m
ay not be secure. If you wish to have extraction utilities executed as the curre
nt user, use '--run-as=root' (binwalk itself must be run as root).
------------------------------------------------------------------------------
------------------
Traceback (most recent call last):
  File "/usr/lib/python3/dist-packages/binwalk/core/module.py", line 258, in __i
nit__
    self.load()
  File "/usr/lib/python3/dist-packages/binwalk/modules/extractor.py", line 147,
in load
    raise ModuleException("Binwalk extraction uses many third party utilities, w
hich may not be secure. If you wish to have extraction utilities executed as the
 current user, use '--run-as=%s' (binwalk itself must be run as root)." % user_i
nfo.pw_name)
binwalk.core.exceptions.ModuleException: Binwalk extraction uses many third part
y utilities, which may not be secure. If you wish to have extraction utilities e
xecuted as the current user, use '--run-as=root' (binwalk itself must be run as
root).
------------------------------------------------------------------------------
------------------

luffy@luff-vmware:~/pyew$
```

Fig. 33

After successful extraction, the verification chain showed:

Shim ⟶ GRUB ⟶ Kernel verification:

Shim (shimx64.efi)

Trusted by Microsoft UEFI CA (Task 2 db variable)

Embedded certificate X (Canonical Secure Boot Signing)

GRUB (grubx64.efi)

Verified by Shim using certificate X

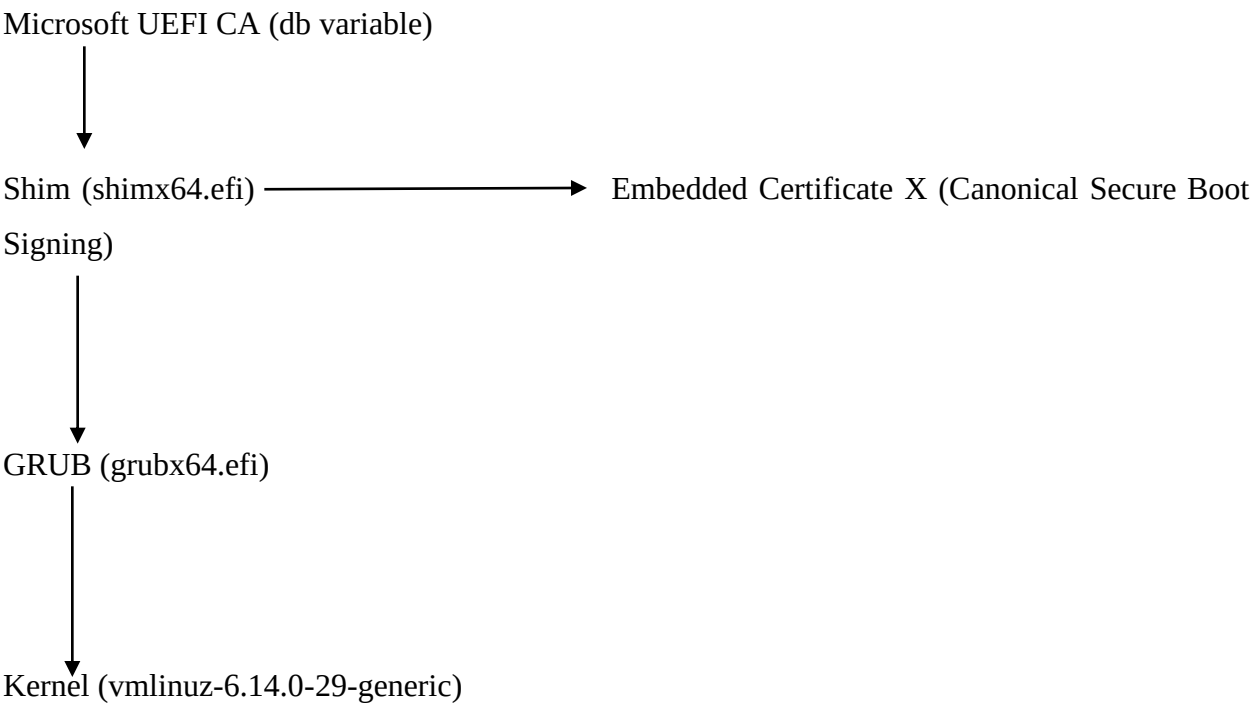
Signed by Canonical Ltd. Secure Boot Signing (2022 v1)

Kernel (vmlinuz-6.14.0-29-generic)

GRUB verifies kernel signature using same certificate X

Kernel is trusted and boots securely

Diagram (Chain of Trust):



Part 13 of Task 3: Verify that certificate X signed the GRUB binary

Command [14]:

sudo sbverify --list /boot/efi/EFI/ubuntu/grubx64.efi

Output (Fig. 34):

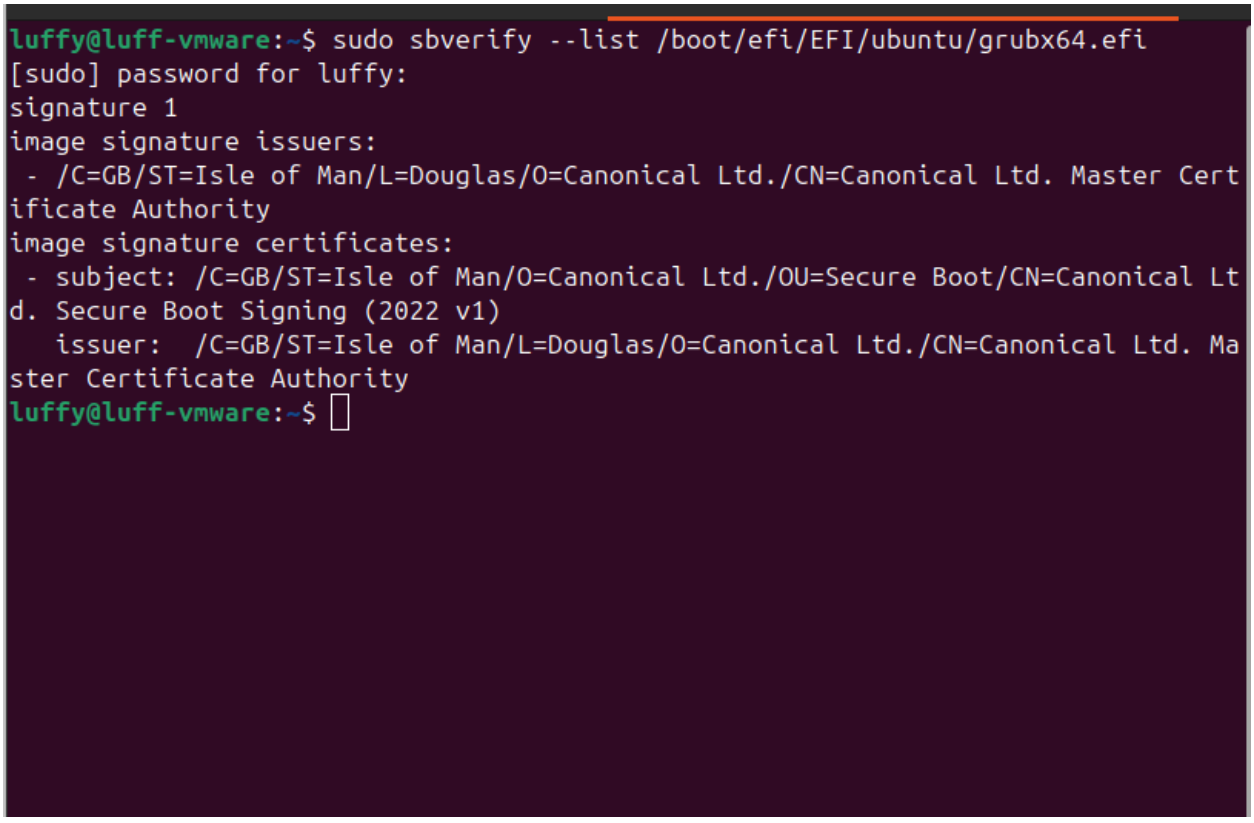


Fig. 34

The GRUB binary in the output is indeed signed with a Canonical Ltd. Secure Boot Signing (2022 v1) certificate.

Issuer: Canonical Ltd. Master Certificate Authority

Subject: Canonical Ltd. Secure Boot Signing (2022 v1)

The certificate X embedded in the shim is Canonical Ltd. Secure Boot Signing (2022 v1), and it is the one used to sign the GRUB binary. The GRUB binary is signed with the corresponding private key of certificate X **[14].**

Task 4 - The Kernel

GRUB allows the user to select from a list of kernels. According to Step 3, GRUB will check if the chosen kernel is signed. If so, GRUB leaves the system in UEFI mode before loading and running the kernel, so that it still has access to the UEFI services. As with shim, GRUB needs a trusted certificate against which it can verify the signed kernel. This is the same certificate as the *shim* uses.

14. Verify the kernel you booted against the X certificate.

15. Where does GRUB get its trusted certificate from? Hint: It is not stored in the binary, and it is not stored on the file system.

16. Draw a diagram that shows the chain of trust from the UEFI PK key to the signed kernel. Show all certificates, binaries, and signing relations involved.

Answer:

Part 14 of Task 4: Verify the kernel you booted against the X certificate

Command [14]:

ls -lh /boot/vmlinuz-6.14.0-29-generic

Output (Fig. 35):

```
luffy@luff-vmware:~$ ls -lh /boot/vmlinuz-6.14.0-29-generic
-rw------- 1 root root 15M Aug 14 21:08 /boot/vmlinuz-6.14.0-29-generic
luffy@luff-vmware:~$
```

Fig. 35

Command [14]:

sudo pesign -S -i /boot/vmlinuz-6.14.0-29-generic -v

Output: (Fig. 36)

```
luffy@luff-vmware:~$ sudo pesign -S -i /boot/vmlinuz-6.14.0-29-generic -v
---------------------------------------------
certificate address is 0x77bb492d3208
Content was not encrypted.
Content is detached; signature cannot be verified.
The signer's common name is Canonical Ltd. Secure Boot Signing (2022 v1)
No signer email address.
Signing time: Sat Aug 16, 2025
There were certs or crls included.
---------------------------------------------
luffy@luff-vmware:~$
```

Fig. 36

Command [14]:

sudo sbverify --list /boot/vmlinuz-6.14.0-29-generic

Output (Fig. 37):

```
luffy@luff-vmware:~$ sudo sbverify --list /boot/vmlinuz-6.14.0-29-generic
signature 1
image signature issuers:
 - /C=GB/ST=Isle of Man/L=Douglas/O=Canonical Ltd./CN=Canonical Ltd. Master Cert
ificate Authority
image signature certificates:
 - subject: /C=GB/ST=Isle of Man/O=Canonical Ltd./OU=Secure Boot/CN=Canonical Lt
d. Secure Boot Signing (2022 v1)
   issuer:  /C=GB/ST=Isle of Man/L=Douglas/O=Canonical Ltd./CN=Canonical Ltd. Ma
ster Certificate Authority
luffy@luff-vmware:~$
```

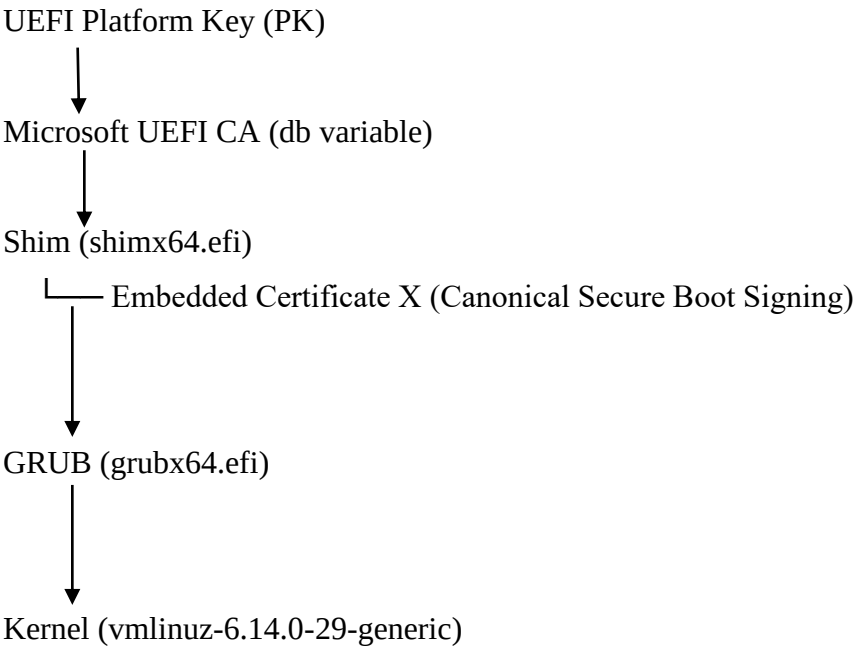
Fig. 37

The kernel is signed with the **Canonical Ltd. Secure Boot Signing (2022 v1)** certificate, which is certificate X. Kernel trusted **[14].**

Part 15 of Task 4: Where does GRUB get its trusted certificate from

GRUB obtains its trusted certificate from the shim binary. Certificate X (Canonical Ltd. Secure Boot Signing) is embedded in the shim. It is not stored in a separate file system location or in the GRUB binary **[14].**

Part 16 of Task 4: Chain of trust diagram **[15]**



In chain of trust:

- **PK (Platform Key):** Root of trust, embedded hardware
- **Microsoft UEFI CA:** db variable, which Shim verifies.
- **Shim:** First stage bootloader, embed certificate X.
- **GRUB:** Verified by Shim, and GRUB kernel verified with certificate X.
- **Kernel:** Signed by Canonical Ltd. Secure Boot Signing (2022 v1), trusted **[15]**.

References

[1] UEFI Secure Boot - ECI Documentation, “mokutil --sb-state SecureBoot enabled,” Intel ECI, Available: https://eci.intel.com/docs/3.0.1/development/secure_boot.html.

[2] “Verify whether a Linux instance is enabled for UEFI Secure Boot,” Amazon Web Services Documentation, Available: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/verify-uefi-secure-boot.html.

[3] Linux Manual Pages, “efi-readvar(8) - manipulate UEFI signature lists,” Linux.die.net, Available: https://linux.die.net/man/8/efi-readvar.

[4] Microsoft Docs, “Secure Boot and UEFI,” Microsoft Learn, Available: https://learn.microsoft.com/en-us/windows-hardware/design/device-experiences/oem-secure-boot.

[5] OpenSSL Project, “openssl-x509(1) - display or manipulate X.509 certificates,” OpenSSL Documentation, Available: https://docs.openssl.org/man1/openssl-x509.html.

[6] Ubuntu Documentation, “UEFI/SecureBoot/Testing - Microsoft signs Canonical’s ‘shim’ with their ‘Microsoft Corporation UEFI CA’,” Ubuntu Wiki, Available: https://wiki.ubuntu.com/UEFI/SecureBoot/Testing.

[7] Amazon Web Services Documentation, “How UEFI Secure Boot works with Amazon EC2 instances,” AWS User Guide, available: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/how-uefi-secure-boot-works.html.

[8] Ubuntu Documentation, “Secure Boot - Shim Bootloader in Ubuntu,” Ubuntu Wiki, Available: https://ubuntu.com/security/secure-boot.

[9] Microsoft Corp., “Windows Secure Boot certificates expire in June 2026 - Act now,” Microsoft Tech Community Blog, 26 June 2025, Available: https://techcommunity.microsoft.com/blog/windows-itpro-blog/act-now-secure-boot-certificates-expire-in-june-2026/.

[10] Microsoft, “Microsoft PE and COFF Specification - Attribute Certificates (IMAGE_DIRECTORY_ENTRY_SECURITY),” Microsoft Learn, Available: https://learn.microsoft.com/en-us/windows/win32/debug/pe-format#the-attribute-certificate-table-image-only.

[11] OpenSSL Project, “openssl-pkcs7(1) - PKCS#7 command line utility,” OpenSSL Documentation, [online], Available: https://docs.openssl.org/man1/openssl-pkcs7.html.

[12] R. Kolmogorov, "sbverify - verify UEFI Secure Boot signatures," rhboot GitHub Documentation, , Available: https://github.com/rhboot/sbsigntools.

[13] Canonical, “Shim - a UEFI boot loader that handles secure boot,” Ubuntu Wiki, Available: https://wiki.ubuntu.com/UEFI/SecureBoot/Testing.

[14] R. Kolmogorov, “sbverify - verify UEFI Secure Boot signatures,” rhboot GitHub Documentation, Available: https://github.com/rhboot/sbsigntools.

[15] Canonical, “UEFI Secure Boot - Chain of Trust in Ubuntu,” Ubuntu Documentation, Available: https://wiki.ubuntu.com/UEFI/SecureBoot.